diff --git a/BACKTESTING.md b/BACKTESTING.md

new file mode 100644

index 0000000000000000000000000000000000000000..2d88235e885e8bb2d821720a7c2626a0571d6dd9

--- /dev/null

+++ b/BACKTESTING.md

@@ -0,0 +1,55 @@

+# ICT NQ Backtester (v3 foundation)

+

+This repository now includes `backtest_ict_nq.py`, a modular backtesting engine designed as a foundation that can later be connected to live execution.

+

+## Improvements included from the prior simulation review

+

+- **DST-safe kill-zone handling** (timezone-safe if enabled), but default now trades all sessions.

+- **Dynamic risk tiers** with capped max risk.

+- **ATR-expansion trend analysis** and projected-move gating.

+- **Regime gating for aggressive tiers** (`STRONG`/`ELITE`) to reduce over-sizing in chop.

+- **Slippage + commission modeling** for more realistic PnL.

+- **Daily risk guardrails**:

+ - max daily loss lock,

+ - max consecutive-loss lock,

+ - max trades per day cap,

+ - post-exit cooldown bars to prevent overtrading.

+- **Early de-risk criterion**: if profit is acceptable but continuation looks uncertain, it takes partial profits and tightens stop.

+- **Time stop**: recycles capital if a trade stalls too long.

+- **Modular architecture** (signals, trend analysis, risk model, execution model) to support migration to a market-trading runtime.

+

+## Input data format

+

+Both files (`--bars5m`, `--bars1h`) must be CSV with headers:

+

+```text

+timestamp,open,high,low,close

+```

+

+- `timestamp` should be ISO-8601 (e.g. `2026-01-05T14:35:00Z`).

+- Naive timestamps are treated as UTC.

+

+## Run

+

+```bash

+python3 backtest_ict_nq.py --bars5m data/nq_5m.csv --bars1h data/nq_1h.csv

+```

+

+## Notes for live-trading migration

+

+To turn this into a live trading bot safely, keep this separation:

+

+1. **Data adapter** (Tradovate stream/websocket)

+2. **Strategy engine** (existing `Backtester` logic refactored into an incremental evaluator)

+3. **Execution adapter** (order placement, cancel/replace, fill reconciliation)

+4. **Risk supervisor** (daily locks, drawdown, session lockouts)

+5. **Telemetry** (trade journal + hourly Telegram)

+

+This way, your decision logic remains testable while transport and broker logic can be swapped independently.

+

+

+## New default behavior

+

+- `trade_only_kill_zone = False` (bot can trade any time).

+- Trade frequency is regulated with `max_trades_per_day = 6` and `cooldown_bars_after_exit = 3`.

+- `min_score = 25` to increase signal frequency while still filtered by risk guardrails.

diff --git a/backtest_ict_nq.py b/backtest_ict_nq.py

new file mode 100644

index 0000000000000000000000000000000000000000..82cfa2bf15ec3aa1b34aa2174d8abdf59ea1393b

--- /dev/null

+++ b/backtest_ict_nq.py

@@ -0,0 +1,687 @@

+from __future__ import annotations

+

```
+import argparse
+import csv
+import math
+from collections import Counter
+from dataclasses import dataclass, field
+from datetime import datetime, time
+from pathlib import Path
+from typing import Dict, List, Optional, Tuple
+from zoneinfo import ZoneInfo
+
+NY_TZ = ZoneInfo("America/New_York")
+UTC_TZ = ZoneInfo("UTC")
+
+
+@dataclass
+class Bar:
+    ts: datetime  # UTC
+    open: float
+    high: float
+    low: float
+    close: float
+
+
+@dataclass
+class RiskTier:
+    min_score: int
```

```
+    risk_pct: float
+    label: str
+
+
+@dataclass
+class StrategyConfig:
+    symbol: str = "NQ"
+    account_size: float = 50_000.0
+
+    max_risk_pct: float = 0.02
+    risk_tiers: List[RiskTier] = field(
+        default_factory=lambda: [
+            RiskTier(90, 0.020, "ELITE"),
+            RiskTier(75, 0.015, "STRONG"),
+            RiskTier(60, 0.010, "MODERATE"),
+            RiskTier(50, 0.005, "STANDARD"),
+        ]
+    )
+
+    atr_period: int = 14
+    atr_multiplier_sl: float = 1.5
+    atr_expansion_window: int = 5
+    atr_expansion_threshold: float = 1.3
+
+    tp1_ratio: float = 1.5
+    tp2_ratio: float = 2.5
```

```
+   tp3_ratio: float = 4.0
+   tp1_size: float = 0.40
+   tp2_size: float = 0.40
+
+   ob_lookback: int = 10
+   fvg_min_size: float = 5.0
+   liq_lookback: int = 20
+   mss_confirm_bars: int = 2
+   min_score: int = 25
+
+   trade_only_kill_zone: bool = False
+   kill_zones: Tuple[Tuple[time, time], ...] = (
+       (time(3, 0), time(5, 0)),
+       (time(9, 30), time(11, 0)),
+       (time(13, 30), time(15, 0)),
+   )
+
+   slippage_ticks: int = 2
+   commission_per_contract: float = 2.5
+   tick_size: float = 0.25
+   tick_value: float = 5.0
+
+   max_daily_loss: float = 1500.0
+   max_consecutive_losses: int = 3
+   require_regime_for_aggressive_tiers: bool = True
+   max_trades_per_day: int = 6
```

```
+    cooldown_bars_after_exit: int = 3
+    max_bars_in_trade: int = 48
+    de_risk_min_r_multiple: float = 1.0
+    de_risk_close_pct: float = 0.50
+    de_risk_lock_r_multiple: float = 0.35
+    uncertainty_lookback_bars: int = 3
+
+
+@dataclass
+class Signal:
+    direction: Optional[str]
+    score: int
+    reasons: List[str]
+
+
+@dataclass
+class Position:
+    is_long: bool
+    qty: int
+    remaining_qty: int
+    entry: float
+    initial_stop: float
+    stop: float
+    tp1: float
+    tp2: float
+    tp3: float
```

```
+    tier: str
+    score: int
+    opened_at: datetime
+    opened_bar_idx: int
+    tp1_hit: bool = False
+    tp2_hit: bool = False
+    de_risk_done: bool = False
+
+
+@dataclass
+class TradeLog:
+    opened_at: datetime
+    closed_at: datetime
+    direction: str
+    qty: int
+    score: int
+    tier: str
+    gross_pnl: float
+    fees: float
+    net_pnl: float
+    r_multiple: float
+    reason: str
+
+
+class Backtester:
+    def __init__(self, cfg: StrategyConfig, bars_5m: List[Bar], bars_1h: List[Bar]):
```

```
+        self.cfg = cfg
+        self.bars_5m = bars_5m
+        self.bars_1h = bars_1h
+        self.equity = cfg.account_size
+
+        self.position: Optional[Position] = None
+        self.logs: List[TradeLog] = []
+        self.reject_reasons: Counter = Counter()
+
+        self.current_day = None
+        self.day_start_equity = self.equity
+        self.consecutive_losses = 0
+        self.day_locked = False
+        self.trades_opened_today = 0
+        self.last_exit_bar_idx = -10_000
+
+        self._active_trade_gross = 0.0
+        self._active_trade_fees = 0.0
+        self.trades_opened_today += 1
+
+    # ---------- indicators ----------
+    def calc_atr(self, bars: List[Bar], period: int) -> Optional[float]:
+        if len(bars) < period + 1:
+            return None
+        trs = []
+        for i in range(len(bars) - period, len(bars)):
```

```
+        h, l, pc = bars[i].high, bars[i].low, bars[i - 1].close
+        trs.append(max(h - l, abs(h - pc), abs(l - pc)))
+    return sum(trs) / period
+
+ def get_htf_bias(self, htf_slice: List[Bar]) -> str:
+    if len(htf_slice) < 21:
+        return "NEUTRAL"
+    ema = htf_slice[0].close
+    k = 2 / 21
+    for b in htf_slice[1:]:
+        ema = b.close * k + ema * (1 - k)
+    last = htf_slice[-1].close
+    if last > ema * 1.002:
+        return "BULLISH"
+    if last < ema * 0.998:
+        return "BEARISH"
+    return "NEUTRAL"
+
+ def get_htf_slope(self, htf_slice: List[Bar], lookback: int = 8) -> float:
+    if len(htf_slice) < lookback:
+        return 0.0
+    return htf_slice[-1].close - htf_slice[-lookback].close
+
+ def detect_order_blocks(self, bars: List[Bar]) -> Dict[str, Optional[Dict[str, float]]]:
+    out = {"bullish": None, "bearish": None}
+    if len(bars) < self.cfg.ob_lookback + 3:
```

```
+        return out
+    for i in range(len(bars) - self.cfg.ob_lookback, len(bars) - 2):
+        c, n, n2 = bars[i], bars[i + 1], bars[i + 2]
+        if c.close < c.open and n.close > c.high and n2.close > n.high:
+            out["bullish"] = {"high": c.high, "low": c.low}
+        if c.close > c.open and n.close < c.low and n2.close < n.low:
+            out["bearish"] = {"high": c.high, "low": c.low}
+    return out
+
+ def detect_fvg(self, bars: List[Bar]) -> Dict[str, Optional[Dict[str, float]]]:
+    out = {"bullish": None, "bearish": None}
+    if len(bars) < 4:
+        return out
+    for i in range(len(bars) - 4, len(bars) - 2):
+        prev, nxt = bars[i], bars[i + 2]
+        if nxt.low - prev.high >= self.cfg.fvg_min_size:
+            out["bullish"] = {"bottom": prev.high, "top": nxt.low}
+        if prev.low - nxt.high >= self.cfg.fvg_min_size:
+            out["bearish"] = {"bottom": nxt.high, "top": prev.low}
+    return out
+
+ def detect_liquidity_sweep(self, bars: List[Bar]) -> Dict[str, Optional[float]]:
+    out = {"swept_high": False, "swept_low": False, "high_level": None, "low_level": None}
+    if len(bars) < self.cfg.liq_lookback + 2:
+        return out
+    hi = max(b.high for b in bars[-self.cfg.liq_lookback - 1 : -1])
```

```
+     lo = min(b.low for b in bars[-self.cfg.liq_lookback - 1 : -1])
+     c = bars[-1]
+     if c.high > hi and c.close < hi:
+         out["swept_high"], out["high_level"] = True, hi
+     if c.low < lo and c.close > lo:
+         out["swept_low"], out["low_level"] = True, lo
+     return out
+
+   def detect_mss(self, bars: List[Bar]) -> Optional[str]:
+     n = len(bars)
+     if n < 10:
+         return None
+     swing_high, swing_low = -math.inf, math.inf
+     for i in range(n - 8, n - self.cfg.mss_confirm_bars):
+         b = bars[i]
+         if b.high > bars[i - 1].high and b.high > bars[i + 1].high:
+             swing_high = b.high
+         if b.low < bars[i - 1].low and b.low < bars[i + 1].low:
+             swing_low = b.low
+
+     above = swing_high != -math.inf and all(
+         bars[i].close > swing_high for i in range(n - self.cfg.mss_confirm_bars, n)
+     )
+     below = swing_low != math.inf and all(
+         bars[i].close < swing_low for i in range(n - self.cfg.mss_confirm_bars, n)
+     )
```

```
+     if above:
+         return "BULLISH_MSS"
+     if below:
+         return "BEARISH_MSS"
+     return None
+
+   def score_setup(self, bias: str, ob: dict, fvg: dict, liq: dict, mss: Optional[str]) -> Signal:
+     score, reasons = 0, []
+     if bias == "BULLISH":
+         if ob["bullish"]:
+             score += 25; reasons.append("Bullish OB")
+         if fvg["bullish"]:
+             score += 25; reasons.append("Bullish FVG")
+         if liq["swept_low"]:
+             score += 25; reasons.append("Buy-side liquidity sweep")
+         if mss == "BULLISH_MSS":
+             score += 25; reasons.append("Bullish MSS")
+         return Signal("LONG" if score >= self.cfg.min_score else None, score, reasons)
+
+     if bias == "BEARISH":
+         if ob["bearish"]:
+             score += 25; reasons.append("Bearish OB")
+         if fvg["bearish"]:
+             score += 25; reasons.append("Bearish FVG")
+         if liq["swept_high"]:
+             score += 25; reasons.append("Sell-side liquidity sweep")
```

```
+        if mss == "BEARISH_MSS":
+            score += 25; reasons.append("Bearish MSS")
+        return Signal("SHORT" if score >= self.cfg.min_score else None, score, reasons)
+
+    return Signal(None, score, reasons)
+
+  def analyze_trend(self, ltf: List[Bar], htf: List[Bar], signal: Signal) -> Dict[str, float | bool]:
+    atr = self.calc_atr(ltf, self.cfg.atr_period) or 10.0
+    recent = 0.0
+    if len(ltf) >= self.cfg.atr_expansion_window + 1:
+        tr_sum = 0.0
+        for i in range(len(ltf) - self.cfg.atr_expansion_window, len(ltf)):
+            h, l, pc = ltf[i].high, ltf[i].low, ltf[i - 1].close
+            tr_sum += max(h - l, abs(h - pc), abs(l - pc))
+        recent = tr_sum / self.cfg.atr_expansion_window
+
+    expansion = (recent / atr) if recent > 0 else 1.0
+    trending = expansion >= self.cfg.atr_expansion_threshold
+    projected = atr * self.cfg.tp2_ratio * (1 + signal.score / 100) * (expansion if trending
else 1.0)
+
+    return {
+        "atr": atr,
+        "atr_expansion": expansion,
+        "is_trending": trending,
+        "projected_move": projected,
```

```
+        "htf_slope": self.get_htf_slope(htf),
+    }
+
+ # ---------- risk ----------
+ def in_kill_zone(self, ts_utc: datetime) -> bool:
+    if not self.cfg.trade_only_kill_zone:
+        return True
+    local_t = ts_utc.astimezone(NY_TZ).time()
+    return any(start <= local_t <= end for start, end in self.cfg.kill_zones)
+
+ def day_guardrails_allow_new_trade(self, ts_utc: datetime) -> bool:
+    day = ts_utc.astimezone(NY_TZ).date()
+    if day != self.current_day:
+        self.current_day = day
+        self.day_start_equity = self.equity
+        self.day_locked = False
+        self.consecutive_losses = 0
+        self.trades_opened_today = 0
+
+    daily_pnl = self.equity - self.day_start_equity
+    if daily_pnl <= -abs(self.cfg.max_daily_loss):
+        self.day_locked = True
+        self.reject_reasons["daily_loss_lock"] += 1
+    if self.consecutive_losses >= self.cfg.max_consecutive_losses:
+        self.day_locked = True
+        self.reject_reasons["consecutive_loss_lock"] += 1
```

```
+        if self.trades_opened_today >= self.cfg.max_trades_per_day:
+            self.reject_reasons["max_trades_per_day"] += 1
+            return False
+        return not self.day_locked
+
+    def pick_tier(self, score: int) -> RiskTier:
+        for tier in self.cfg.risk_tiers:
+            if score >= tier.min_score:
+                return tier
+        return self.cfg.risk_tiers[-1]
+
+    def allow_aggressive_tier(self, tier: RiskTier, trend: dict, signal: Signal) -> bool:
+        if not self.cfg.require_regime_for_aggressive_tiers or tier.label not in {"STRONG",
"ELITE"}:
+            return True
+        slope = trend["htf_slope"]
+        direction_ok = (signal.direction == "LONG" and slope > 0) or (signal.direction ==
"SHORT" and slope < 0)
+        ok = bool(trend["is_trending"] and direction_ok)
+        if not ok:
+            self.reject_reasons["aggressive_tier_downgraded"] += 1
+        return ok
+
+    def size_position(self, score: int, atr: float, trend: dict, signal: Signal) -> Tuple[int,
RiskTier, float]:
+        tier = self.pick_tier(score)
+        if not self.allow_aggressive_tier(tier, trend, signal):
```

```
+        tier = RiskTier(60, 0.010, "MODERATE") if score >= 60 else RiskTier(50, 0.005, "STANDARD")
+
+     expansion_mult = min(1.5, trend["atr_expansion"]) if trend["is_trending"] else 1.0
+     risk_pct = min(tier.risk_pct * expansion_mult, self.cfg.max_risk_pct)
+
+     risk_amount = self.equity * risk_pct
+     sl_distance = atr * self.cfg.atr_multiplier_sl
+     sl_ticks = max(1, round(sl_distance / self.cfg.tick_size))
+     risk_per_contract = sl_ticks * self.cfg.tick_value
+     qty = max(1, int(risk_amount // risk_per_contract))
+     return qty, tier, sl_distance
+
+   # ---------- execution ----------
+   def apply_slippage(self, price: float, is_long: bool, is_entry: bool) -> float:
+     slip = self.cfg.slippage_ticks * self.cfg.tick_size
+     if is_entry:
+        return price + slip if is_long else price - slip
+     return price - slip if is_long else price + slip
+
+   def pnl_for_leg(self, entry: float, exit_price: float, qty: int, is_long: bool) -> Tuple[float, float, float]:
+     signed_points = (exit_price - entry) if is_long else (entry - exit_price)
+     ticks = signed_points / self.cfg.tick_size
+     gross = ticks * self.cfg.tick_value * qty
+     fees = self.cfg.commission_per_contract * qty
+     net = gross - fees
```

```
+       return gross, fees, net
+
+   def open_position(self, bar: Bar, bar_idx: int, signal: Signal, ob: dict, fvg: dict, trend: dict):
+       is_long = signal.direction == "LONG"
+       ref = (
+           (ob["bullish"]["high"] if ob["bullish"] else (fvg["bullish"]["bottom"] if fvg["bullish"] else bar.close))
+           if is_long
+           else (ob["bearish"]["low"] if ob["bearish"] else (fvg["bearish"]["top"] if fvg["bearish"] else bar.close))
+       )
+
+       qty, tier, sl_dist = self.size_position(signal.score, float(trend["atr"]), trend, signal)
+       tp_mult = min(1.5, float(trend["atr_expansion"])) if trend["is_trending"] else 1.0
+
+       entry = self.apply_slippage(ref, is_long=is_long, is_entry=True)
+       stop = entry - sl_dist if is_long else entry + sl_dist
+       tp1 = entry + sl_dist * self.cfg.tp1_ratio if is_long else entry - sl_dist * self.cfg.tp1_ratio
+       tp2 = entry + sl_dist * self.cfg.tp2_ratio * tp_mult if is_long else entry - sl_dist * self.cfg.tp2_ratio * tp_mult
+       tp3 = entry + sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2 if is_long else entry - sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2
+
+       self.position = Position(
+           is_long=is_long,
+           qty=qty,
```

```
+         remaining_qty=qty,
+         entry=entry,
+         initial_stop=stop,
+         stop=stop,
+         tp1=tp1,
+         tp2=tp2,
+         tp3=tp3,
+         tier=tier.label,
+         score=signal.score,
+         opened_at=bar.ts,
+         opened_bar_idx=bar_idx,
+     )
+     self._active_trade_gross = 0.0
+     self._active_trade_fees = 0.0
+     self.trades_opened_today += 1
+
+   def close_leg(self, bar: Bar, target_price: float, qty: int):
+     assert self.position is not None
+     p = self.position
+     qty = max(0, min(qty, p.remaining_qty))
+     if qty == 0:
+         return
+     exit_price = self.apply_slippage(target_price, is_long=p.is_long, is_entry=False)
+     gross, fees, net = self.pnl_for_leg(p.entry, exit_price, qty, p.is_long)
+     self.equity += net
+     self._active_trade_gross += gross
```

```
+        self._active_trade_fees += fees
+        p.remaining_qty -= qty
+
+    def finalize_trade(self, bar: Bar, reason: str, bar_idx: int):
+        assert self.position is not None
+        p = self.position
+        net = self._active_trade_gross - self._active_trade_fees
+        risk_per_contract = abs(p.entry - p.initial_stop) / self.cfg.tick_size * self.cfg.tick_value
+        total_risk = max(1.0, risk_per_contract * p.qty)
+        r_mult = net / total_risk
+        self.logs.append(
+            TradeLog(
+                opened_at=p.opened_at,
+                closed_at=bar.ts,
+                direction="LONG" if p.is_long else "SHORT",
+                qty=p.qty,
+                score=p.score,
+                tier=p.tier,
+                gross_pnl=self._active_trade_gross,
+                fees=self._active_trade_fees,
+                net_pnl=net,
+                r_multiple=r_mult,
+                reason=reason,
+            )
+        )
+        if net < 0:
```

```
+        self.consecutive_losses += 1
+    else:
+        self.consecutive_losses = 0
+    self.position = None
+    self.last_exit_bar_idx = bar_idx
+
+  def _is_uncertain_continuation(self, is_long: bool, bar_idx: int) -> bool:
+    lb = self.cfg.uncertainty_lookback_bars
+    if bar_idx < lb:
+        return False
+    closes = [self.bars_5m[i].close for i in range(bar_idx - lb, bar_idx + 1)]
+    up_count = sum(1 for i in range(1, len(closes)) if closes[i] > closes[i - 1])
+    down_count = sum(1 for i in range(1, len(closes)) if closes[i] < closes[i - 1])
+    if is_long:
+        return down_count >= max(2, lb - 1)
+    return up_count >= max(2, lb - 1)
+
+  def _maybe_de_risk_early(self, p: Position, bar: Bar, bar_idx: int) -> bool:
+    if p.de_risk_done:
+        return False
+    initial_r = abs(p.entry - p.initial_stop)
+    if initial_r <= 0:
+        return False
+    favorable_move = (bar.close - p.entry) if p.is_long else (p.entry - bar.close)
+    r_now = favorable_move / initial_r
+    if r_now < self.cfg.de_risk_min_r_multiple:
```

```
+        return False
+    if not self._is_uncertain_continuation(p.is_long, bar_idx):
+        return False
+
+    close_qty = max(1, round(p.remaining_qty * self.cfg.de_risk_close_pct))
+    self.close_leg(bar, bar.close, close_qty)
+    lock_move = initial_r * self.cfg.de_risk_lock_r_multiple
+    if p.is_long:
+        p.stop = max(p.stop, p.entry + lock_move)
+    else:
+        p.stop = min(p.stop, p.entry - lock_move)
+    p.de_risk_done = True
+    self.reject_reasons["early_de_risk_taken"] += 1
+    if p.remaining_qty <= 0:
+        self.finalize_trade(bar, "EARLY_DERISK_FULL_EXIT", bar_idx)
+        return True
+    return False
+
+  def manage_position(self, bar: Bar, bar_idx: int):
+    if not self.position:
+        return
+    p = self.position
+
+    # Time-stop to recycle capital in slow/sideways conditions.
+    if bar_idx - p.opened_bar_idx >= self.cfg.max_bars_in_trade:
+        self.close_leg(bar, bar.close, p.remaining_qty)
```

```
+        self.finalize_trade(bar, "TIME_STOP", bar_idx)
+        return
+
+    # If a reasonable profit is reached but continuation looks uncertain, de-risk early.
+    if self._maybe_de_risk_early(p, bar, bar_idx):
+        return
+
+    if not p.tp1_hit and ((p.is_long and bar.high >= p.tp1) or (not p.is_long and bar.low <= p.tp1)):
+        p.tp1_hit = True
+        self.close_leg(bar, p.tp1, max(1, round(p.qty * self.cfg.tp1_size)))
+        p.stop = p.entry
+
+    if p.tp1_hit and not p.tp2_hit and ((p.is_long and bar.high >= p.tp2) or (not p.is_long and bar.low <= p.tp2)):
+        p.tp2_hit = True
+        self.close_leg(bar, p.tp2, max(1, round(p.qty * self.cfg.tp2_size)))
+        p.stop = p.tp1
+
+    if p.tp1_hit and p.tp2_hit and ((p.is_long and bar.high >= p.tp3) or (not p.is_long and bar.low <= p.tp3)):
+        self.close_leg(bar, p.tp3, p.remaining_qty)
+        self.finalize_trade(bar, "TP3", bar_idx)
+        return
+
+    if (p.is_long and bar.low <= p.stop) or (not p.is_long and bar.high >= p.stop):
+        self.close_leg(bar, p.stop, p.remaining_qty)
```

```
+        self.finalize_trade(bar, "STOP", bar_idx)
+
+   # ---------- run ----------
+   def run(self) -> Dict[str, float]:
+       h1_cursor = 0
+       h1_slice: List[Bar] = []
+
+       for i, bar in enumerate(self.bars_5m):
+           while h1_cursor < len(self.bars_1h) and self.bars_1h[h1_cursor].ts <= bar.ts:
+               h1_slice.append(self.bars_1h[h1_cursor])
+               h1_cursor += 1
+
+           if self.position:
+               self.manage_position(bar, i)
+               continue
+
+           if i < 30:
+               self.reject_reasons["warmup"] += 1
+               continue
+           if not self.in_kill_zone(bar.ts):
+               self.reject_reasons["outside_kill_zone"] += 1
+               continue
+           if not self.day_guardrails_allow_new_trade(bar.ts):
+               continue
+           if i - self.last_exit_bar_idx <= self.cfg.cooldown_bars_after_exit:
+               self.reject_reasons["cooldown_active"] += 1
```

```
+            continue
+
+        ltf = self.bars_5m[: i + 1]
+        bias = self.get_htf_bias(h1_slice)
+        ob = self.detect_order_blocks(ltf)
+        fvg = self.detect_fvg(ltf)
+        liq = self.detect_liquidity_sweep(ltf)
+        mss = self.detect_mss(ltf)
+        signal = self.score_setup(bias, ob, fvg, liq, mss)
+
+        if not signal.direction or signal.score < self.cfg.min_score:
+            self.reject_reasons["weak_or_no_signal"] += 1
+            continue
+
+        trend = self.analyze_trend(ltf, h1_slice, signal)
+        if trend["projected_move"] < trend["atr"] * 1.1:
+            self.reject_reasons["projected_move_too_small"] += 1
+            continue
+
+        self.open_position(bar, i, signal, ob, fvg, trend)
+
+    # force-close if still open on last bar
+    if self.position:
+        last = self.bars_5m[-1]
+        self.close_leg(last, last.close, self.position.remaining_qty)
+        self.finalize_trade(last, "FORCED_EOD_CLOSE", len(self.bars_5m)-1)
```

```
+
+     net = self.equity - self.cfg.account_size
+     wins = sum(1 for t in self.logs if t.net_pnl > 0)
+     losses = len(self.logs) - wins
+     win_rate = wins / len(self.logs) if self.logs else 0.0
+     profit_factor = (
+         sum(t.net_pnl for t in self.logs if t.net_pnl > 0)
+         / max(1e-9, abs(sum(t.net_pnl for t in self.logs if t.net_pnl < 0)))
+         if self.logs
+         else 0.0
+     )
+
+     return {
+         "starting_equity": self.cfg.account_size,
+         "ending_equity": self.equity,
+         "net_pnl": net,
+         "trades": len(self.logs),
+         "wins": wins,
+         "losses": losses,
+         "win_rate": win_rate,
+         "avg_net_pnl": (net / len(self.logs)) if self.logs else 0.0,
+         "profit_factor": profit_factor,
+     }
+
+ def write_trade_log_csv(self, out_path: Path):
+     out_path.parent.mkdir(parents=True, exist_ok=True)
```

```
+     with out_path.open("w", newline="", encoding="utf-8") as f:
+       w = csv.writer(f)
+       w.writerow([
+         "opened_at", "closed_at", "direction", "qty", "score", "tier",
+         "gross_pnl", "fees", "net_pnl", "r_multiple", "reason",
+       ])
+       for t in self.logs:
+         w.writerow([
+           t.opened_at.isoformat(), t.closed_at.isoformat(), t.direction, t.qty,
+           t.score, t.tier, f"{t.gross_pnl:.2f}", f"{t.fees:.2f}",
+           f"{t.net_pnl:.2f}", f"{t.r_multiple:.4f}", t.reason,
+         ])
+
+   def write_dashboard_md(self, out_path: Path, summary: Dict[str, float]):
+     out_path.parent.mkdir(parents=True, exist_ok=True)
+     top_rejects = self.reject_reasons.most_common(8)
+     top_losses = sorted((t for t in self.logs if t.net_pnl < 0), key=lambda x: x.net_pnl)[:10]
+
+     lines = [
+       "# ICT Backtest Dashboard",
+       "",
+       "## Performance",
+       f"- Starting equity: **${summary['starting_equity']:,.2f}**",
+       f"- Ending equity: **${summary['ending_equity']:,.2f}**",
+       f"- Net PnL: **${summary['net_pnl']:,.2f}**",
+       f"- Trades: **{int(summary['trades'])}**",
```

```
+        f"- Win rate: **{summary['win_rate']:.2%}**",
+        f"- Profit factor: **{summary['profit_factor']:.2f}**",
+        "",
+        "## What went wrong (top rejection causes)",
+    ]
+    if top_rejects:
+        for k, v in top_rejects:
+            lines.append(f"- {k}: {v}")
+    else:
+        lines.append("- No rejections recorded.")
+
+    lines += ["", "## Worst losing trades", "| closed_at | direction | tier | score | net_pnl |
reason |", "|---|---:|---:|---:|---:|---|" ]
+    for t in top_losses:
+        lines.append(f"| {t.closed_at.isoformat()} | {t.direction} | {t.tier} | {t.score} |
${t.net_pnl:,.2f} | {t.reason} |")
+
+    out_path.write_text("\n".join(lines), encoding="utf-8")
+
+
+def parse_dt(s: str) -> datetime:
+   dt = datetime.fromisoformat(s.replace("Z", "+00:00"))
+   if dt.tzinfo is None:
+       dt = dt.replace(tzinfo=UTC_TZ)
+   return dt.astimezone(UTC_TZ)
+
+
```

```
+def load_bars_csv(path: str) -> List[Bar]:
+    out = []
+    with open(path, newline="", encoding="utf-8") as f:
+        reader = csv.DictReader(f)
+        for r in reader:
+            out.append(Bar(
+                ts=parse_dt(r["timestamp"]),
+                open=float(r["open"]),
+                high=float(r["high"]),
+                low=float(r["low"]),
+                close=float(r["close"]),
+            ))
+    out.sort(key=lambda b: b.ts)
+    return out
+
+
+def print_summary(summary: Dict[str, float]):
+    print("=== Backtest Summary ===")
+    for k, v in summary.items():
+        if isinstance(v, float):
+            print(f"{k}: {v:.2%}" if "rate" in k else f"{k}: {v:,.2f}")
+        else:
+            print(f"{k}: {v}")
+
+
+def main():
```

```
+    p = argparse.ArgumentParser(description="ICT NQ backtester with dashboard output")
+    p.add_argument("--bars5m", required=True, help="CSV: timestamp,open,high,low,close")
+    p.add_argument("--bars1h", required=True, help="CSV: timestamp,open,high,low,close")
+    p.add_argument("--dashboard-out", default="dashboard.md", help="Markdown dashboard output path")
+    p.add_argument("--trade-log-out", default="trades.csv", help="Trade log CSV output path")
+    args = p.parse_args()
+
+    bars5m = load_bars_csv(args.bars5m)
+    bars1h = load_bars_csv(args.bars1h)
+    bt = Backtester(StrategyConfig(), bars5m, bars1h)
+    summary = bt.run()
+    print_summary(summary)
+    bt.write_trade_log_csv(Path(args.trade_log_out))
+    bt.write_dashboard_md(Path(args.dashboard_out), summary)
+    print(f"dashboard_written: {args.dashboard_out}")
+    print(f"trade_log_written: {args.trade_log_out}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/backtest_ict_nq_v3.py b/backtest_ict_nq_v3.py
new file mode 100644
```

index 0000000000000000000000000000000000000000..4449fd03eaa3c8fa555cb3415603a446b629826b

--- /dev/null

+++ b/backtest_ict_nq_v3.py

@@ -0,0 +1,217 @@

+from __future__ import annotations

+

+from dataclasses import dataclass

+

+from backtest_ict_nq import Backtester, StrategyConfig, Position

+

+

+@dataclass

+class StrategyConfigV3(StrategyConfig):

+   """Hybrid profile combining v1 participation + v2 quality controls."""

+

+   # Base: still active participation

+   min_score: int = 40

+   trade_only_kill_zone: bool = False

+

+   # Context-aware quality gates

+   min_score_outside_kz: int = 60

+   require_mss_outside_kz: bool = True

+   atr_cap_inside_kz: float = 35.0

+   atr_cap_outside_kz: float = 25.0

+

```
+    # Entry quality filters
+    require_directional_candle_for_low_score: bool = True
+    low_score_directional_threshold: int = 101
+    quick_lock_r_multiple: float = 0.10
+    quick_lock_close_pct: float = 0.90
+    quick_lock_stop_r_multiple: float = 0.10
+
+
+class BacktesterV3(Backtester):
+    def _min_score_for_bar(self, bar) -> int:
+        return self.cfg.min_score if self.in_kill_zone(bar.ts) else self.cfg.min_score_outside_kz
+
+    def _atr_cap_for_bar(self, bar) -> float:
+        return self.cfg.atr_cap_inside_kz if self.in_kill_zone(bar.ts) else
self.cfg.atr_cap_outside_kz
+
+    def _need_mss(self, bar) -> bool:
+        return (not self.in_kill_zone(bar.ts)) and self.cfg.require_mss_outside_kz
+
+    def _entry_checks(self, bar, signal, ob, fvg) -> bool:
+        is_long = signal.direction == "LONG"
+
+        # adaptive directional-candle requirement (stricter on weaker setups)
+        if self.cfg.require_directional_candle_for_low_score and signal.score <
self.cfg.low_score_directional_threshold:
+            min_body = self.cfg.tick_size
+            if is_long and not (bar.close > bar.open + min_body):
```

```
+            self.reject_reasons["entry_bar_direction_wrong"] += 1
+            return False
+        if (not is_long) and not (bar.close < bar.open - min_body):
+            self.reject_reasons["entry_bar_direction_wrong"] += 1
+            return False
+
+    # OB/FVG proximity checks kept from v2
+    if is_long and ob.get("bullish") and not (bar.low <= ob["bullish"]["high"]):
+        self.reject_reasons["ob_price_not_reached"] += 1
+        return False
+    if (not is_long) and ob.get("bearish") and not (bar.high >= ob["bearish"]["low"]):
+        self.reject_reasons["ob_price_not_reached"] += 1
+        return False
+
+    if is_long and fvg.get("bullish") and not (bar.low <= fvg["bullish"]["top"]):
+        self.reject_reasons["fvg_price_not_reached"] += 1
+        return False
+    if (not is_long) and fvg.get("bearish") and not (bar.high >= fvg["bearish"]["bottom"]):
+        self.reject_reasons["fvg_price_not_reached"] += 1
+        return False
+
+    # conflicting OB block retained
+    if is_long and ob.get("bearish") and ob["bearish"]["low"] <= bar.close * 1.005:
+        self.reject_reasons["conflicting_ob_blocked"] += 1
+        return False
+    if (not is_long) and ob.get("bullish") and ob["bullish"]["high"] >= bar.close * 0.995:
```

```
+        self.reject_reasons["conflicting_ob_blocked"] += 1
+        return False
+    return True
+
+  def open_position(self, bar, bar_idx, signal, ob, fvg, trend):
+    if not self._entry_checks(bar, signal, ob, fvg):
+        return
+
+    is_long = signal.direction == "LONG"
+    qty, tier, sl_dist = self.size_position(signal.score, float(trend["atr"]), trend, signal)
+    tp_mult = min(1.5, float(trend["atr_expansion"])) if trend["is_trending"] else 1.0
+
+    # v2 improvement retained: anchor to current bar close
+    entry = self.apply_slippage(bar.close, is_long=is_long, is_entry=True)
+    stop = entry - sl_dist if is_long else entry + sl_dist
+    tp1 = entry + sl_dist * self.cfg.tp1_ratio if is_long else entry - sl_dist * self.cfg.tp1_ratio
+    tp2 = entry + sl_dist * self.cfg.tp2_ratio * tp_mult if is_long else entry - sl_dist *
self.cfg.tp2_ratio * tp_mult
+    tp3 = entry + sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2 if is_long else entry - sl_dist *
self.cfg.tp3_ratio * tp_mult * 1.2
+
+    self.position = Position(
+        is_long=is_long,
+        qty=qty,
+        remaining_qty=qty,
+        entry=entry,
+        initial_stop=stop,
```

```
+        stop=stop,
+        tp1=tp1,
+        tp2=tp2,
+        tp3=tp3,
+        tier=tier.label,
+        score=signal.score,
+        opened_at=bar.ts,
+        opened_bar_idx=bar_idx,
+      )
+      self._active_trade_gross = 0.0
+      self._active_trade_fees = 0.0
+      self.trades_opened_today += 1
+
+    def run(self):
+      h1_cursor = 0
+      h1_slice = []
+
+      for i, bar in enumerate(self.bars_5m):
+        while h1_cursor < len(self.bars_1h) and self.bars_1h[h1_cursor].ts <= bar.ts:
+          h1_slice.append(self.bars_1h[h1_cursor])
+          h1_cursor += 1
+
+        if self.position:
+          self.manage_position(bar, i)
+          continue
+
```

```
+        if i < 30:
+            self.reject_reasons["warmup"] += 1
+            continue
+        # still allow all sessions, but with stricter thresholds outside kill-zone
+        if not self.day_guardrails_allow_new_trade(bar.ts):
+            continue
+        if i - self.last_exit_bar_idx <= self.cfg.cooldown_bars_after_exit:
+            self.reject_reasons["cooldown_active"] += 1
+            continue
+
+        ltf = self.bars_5m[: i + 1]
+        bias = self.get_htf_bias(h1_slice)
+        ob = self.detect_order_blocks(ltf)
+        fvg = self.detect_fvg(ltf)
+        liq = self.detect_liquidity_sweep(ltf)
+        mss = self.detect_mss(ltf)
+        signal = self.score_setup(bias, ob, fvg, liq, mss)
+
+        min_score = self._min_score_for_bar(bar)
+        if not signal.direction or signal.score < min_score:
+            self.reject_reasons["weak_or_no_signal"] += 1
+            continue
+
+        if self._need_mss(bar) and mss is None:
+            self.reject_reasons["no_mss_confirmation"] += 1
+            continue
```

```
+
+        trend = self.analyze_trend(ltf, h1_slice, signal)
+        if trend["atr"] > self._atr_cap_for_bar(bar):
+            self.reject_reasons["atr_too_high"] += 1
+            continue
+        if trend["projected_move"] < trend["atr"] * 1.1:
+            self.reject_reasons["projected_move_too_small"] += 1
+            continue
+
+        # enforce slope alignment for all v3 entries to improve hit quality
+        if self.cfg.require_regime_for_aggressive_tiers:
+            slope = trend["htf_slope"]
+            slope_ok = (signal.direction == "LONG" and slope > 0) or (signal.direction ==
"SHORT" and slope < 0)
+            if not slope_ok:
+                self.reject_reasons["slope_misaligned"] += 1
+                continue
+
+        self.open_position(bar, i, signal, ob, fvg, trend)
+
+    if self.position:
+        last = self.bars_5m[-1]
+        self.close_leg(last, last.close, self.position.remaining_qty)
+        self.finalize_trade(last, "FORCED_EOD_CLOSE", len(self.bars_5m)-1)
+
+    net = self.equity - self.cfg.account_size
```

```
+     wins = sum(1 for t in self.logs if t.net_pnl > 0)
+     losses = len(self.logs) - wins
+     win_rate = wins / len(self.logs) if self.logs else 0.0
+     profit_factor = (
+         sum(t.net_pnl for t in self.logs if t.net_pnl > 0)
+         / max(1e-9, abs(sum(t.net_pnl for t in self.logs if t.net_pnl < 0)))
+         if self.logs else 0.0
+     )
+     return {
+         "starting_equity": self.cfg.account_size,
+         "ending_equity": self.equity,
+         "net_pnl": net,
+         "trades": len(self.logs),
+         "wins": wins,
+         "losses": losses,
+         "win_rate": win_rate,
+         "avg_net_pnl": (net / len(self.logs)) if self.logs else 0.0,
+         "profit_factor": profit_factor,
+     }
+
+ def manage_position(self, bar, bar_idx):
+     if not self.position:
+         return
+     p = self.position
+
+     # quick-lock: convert early favorable moves into high-probability winners
```

```
+        initial_r = abs(p.entry - p.initial_stop)
+        if initial_r > 0 and (not p.tp1_hit):
+            favorable_move = (bar.close - p.entry) if p.is_long else (p.entry - bar.close)
+            r_now = favorable_move / initial_r
+            if r_now >= self.cfg.quick_lock_r_multiple:
+                close_qty = max(1, round(p.qty * self.cfg.quick_lock_close_pct))
+                self.close_leg(bar, bar.close, close_qty)
+                lock_move = initial_r * self.cfg.quick_lock_stop_r_multiple
+                if p.is_long:
+                    p.stop = max(p.stop, p.entry + lock_move)
+                else:
+                    p.stop = min(p.stop, p.entry - lock_move)
+                p.tp1_hit = True  # avoid repeated quick-lock
+                self.reject_reasons["quick_lock_taken"] += 1
+
+        # continue with base position management (tp2/tp3/stop/time-stop/de-risk)
+        super().manage_position(bar, bar_idx)
diff --git a/backtest_ict_nq_v3_adaptive.py b/backtest_ict_nq_v3_adaptive.py
new file mode 100644
index 0000000000000000000000000000000000000000..88cfdc4d2f0ab9bddac4c28569093c3e5eab91c2
--- /dev/null
+++ b/backtest_ict_nq_v3_adaptive.py
@@ -0,0 +1,101 @@
+from __future__ import annotations
+
```

```
+import json
+from dataclasses import dataclass, asdict
+from pathlib import Path
+from typing import Optional
+
+from backtest_ict_nq_v3 import BacktesterV3, StrategyConfigV3
+
+
+@dataclass
+class AdaptivePolicy:
+    """Lightweight online policy updated from realized trades."""
+
+    min_score_inside_kz: int = 40
+    min_score_outside_kz: int = 60
+    atr_cap_inside_kz: float = 35.0
+    atr_cap_outside_kz: float = 25.0
+    quick_lock_r_multiple: float = 0.10
+    quick_lock_close_pct: float = 0.90
+
+    wins: int = 0
+    losses: int = 0
+    trades_seen: int = 0
+
+    @property
+    def win_rate(self) -> float:
+        total = self.wins + self.losses
```

```
+        return self.wins / total if total else 0.0
+
+    def update_from_trade(self, pnl: float):
+        self.trades_seen += 1
+        if pnl > 0:
+            self.wins += 1
+        else:
+            self.losses += 1
+
+    def adapt(self):
+        """Heuristic adaptation: tighten after drawdown, loosen after sustained quality."""
+        if self.trades_seen < 20:
+            return
+
+        wr = self.win_rate
+        if wr < 0.45:
+            # tighten filters (quality first)
+            self.min_score_inside_kz = min(80, self.min_score_inside_kz + 5)
+            self.min_score_outside_kz = min(90, self.min_score_outside_kz + 5)
+            self.atr_cap_inside_kz = max(20.0, self.atr_cap_inside_kz - 1.0)
+            self.atr_cap_outside_kz = max(15.0, self.atr_cap_outside_kz - 1.0)
+            self.quick_lock_close_pct = min(0.95, self.quick_lock_close_pct + 0.05)
+        elif wr > 0.62:
+            # cautiously loosen to increase participation
+            self.min_score_inside_kz = max(25, self.min_score_inside_kz - 2)
+            self.min_score_outside_kz = max(40, self.min_score_outside_kz - 2)
```

```
+        self.atr_cap_inside_kz = min(55.0, self.atr_cap_inside_kz + 0.5)
+        self.atr_cap_outside_kz = min(40.0, self.atr_cap_outside_kz + 0.5)
+        self.quick_lock_close_pct = max(0.65, self.quick_lock_close_pct - 0.03)
+
+
+class BacktesterV3Adaptive(BacktesterV3):
+    """V3 with online adaptive policy persistence."""
+
+    def __init__(self, cfg: StrategyConfigV3, bars_5m, bars_1h, policy_path: str =
"v3_policy_state.json"):
+        super().__init__(cfg, bars_5m, bars_1h)
+        self.policy_path = Path(policy_path)
+        self.policy = self._load_policy()
+        self._apply_policy_to_config()
+
+    def _load_policy(self) -> AdaptivePolicy:
+        if self.policy_path.exists():
+            data = json.loads(self.policy_path.read_text(encoding="utf-8"))
+            return AdaptivePolicy(**data)
+        return AdaptivePolicy(
+            min_score_inside_kz=self.cfg.min_score,
+            min_score_outside_kz=self.cfg.min_score_outside_kz,
+            atr_cap_inside_kz=self.cfg.atr_cap_inside_kz,
+            atr_cap_outside_kz=self.cfg.atr_cap_outside_kz,
+            quick_lock_r_multiple=self.cfg.quick_lock_r_multiple,
+            quick_lock_close_pct=self.cfg.quick_lock_close_pct,
```

```
+        )
+
+    def _apply_policy_to_config(self):
+        self.cfg.min_score = self.policy.min_score_inside_kz
+        self.cfg.min_score_outside_kz = self.policy.min_score_outside_kz
+        self.cfg.atr_cap_inside_kz = self.policy.atr_cap_inside_kz
+        self.cfg.atr_cap_outside_kz = self.policy.atr_cap_outside_kz
+        self.cfg.quick_lock_r_multiple = self.policy.quick_lock_r_multiple
+        self.cfg.quick_lock_close_pct = self.policy.quick_lock_close_pct
+
+    def _save_policy(self):
+        self.policy_path.write_text(json.dumps(asdict(self.policy), indent=2), encoding="utf-8")
+
+    def run(self):
+        summary = super().run()
+        for t in self.logs:
+            self.policy.update_from_trade(t.net_pnl)
+        self.policy.adapt()
+        self._save_policy()
+        summary["policy_win_rate"] = self.policy.win_rate
+        summary["policy_trades_seen"] = self.policy.trades_seen
+        return summary
diff --git a/backtest_ict_nq_v3_adaptive_fusion.py b/backtest_ict_nq_v3_adaptive_fusion.py
new file mode 100644
```

index 0000000000000000000000000000000000000000..0cd516ab6ad1786cb5fed351f9ec6667a4c8f23e

--- /dev/null

+++ b/backtest_ict_nq_v3_adaptive_fusion.py

@@ -0,0 +1,247 @@

+from __future__ import annotations

+

+from dataclasses import dataclass

+

+from backtest_ict_nq import Position, Signal

+from backtest_ict_nq_v3 import StrategyConfigV3

+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive

+

+

+@dataclass

+class StrategyConfigV3Fusion(StrategyConfigV3):

+   """Fusion profile: ICT confluences + StdDev pullback/premium + OTE zones + adaptive policy."""

+

+   std_window: int = 50

+   z_entry_threshold: float = 1.0

+   ote_lookback: int = 60

+

+   # Balanced fusion: both families contribute equally by default.

+   ict_weight: float = 0.50

+   std_ote_weight: float = 0.50

```
+
+   # Require minimum quality from each family to avoid one-sided entries.
+   min_component_score: int = 10
+   fusion_score_offset: int = 20
+
+
+class BacktesterV3AdaptiveFusion(BacktesterV3Adaptive):
+   def __init__(self, cfg: StrategyConfigV3Fusion, bars_5m, bars_1h, policy_path: str =
"v3_fusion_policy_state.json"):
+       super().__init__(cfg, bars_5m, bars_1h, policy_path=policy_path)
+
+   @property
+   def cfg(self) -> StrategyConfigV3Fusion:  # type: ignore[override]
+       return self.__dict__["cfg"]
+
+   @cfg.setter
+   def cfg(self, value):
+       self.__dict__["cfg"] = value
+
+   def _rolling_mean_std(self, closes):
+       if len(closes) < self.cfg.std_window:
+           return None, None
+       window = closes[-self.cfg.std_window :]
+       mean = sum(window) / len(window)
+       var = sum((x - mean) ** 2 for x in window) / len(window)
+       return mean, var**0.5
```

```
+
+   def _ote_zone(self, bars):
+       if len(bars) < self.cfg.ote_lookback:
+           return None
+       chunk = bars[-self.cfg.ote_lookback :]
+       swing_high = max(b.high for b in chunk)
+       swing_low = min(b.low for b in chunk)
+       rng = swing_high - swing_low
+       if rng <= 0:
+           return None
+
+       long_top = swing_high - 0.62 * rng
+       long_bottom = swing_high - 0.79 * rng
+       short_bottom = swing_low + 0.62 * rng
+       short_top = swing_low + 0.79 * rng
+
+       return {
+           "long_zone": (min(long_bottom, long_top), max(long_bottom, long_top)),
+           "short_zone": (min(short_bottom, short_top), max(short_bottom, short_top)),
+       }
+
+   def score_std_ote_component(self, ltf, htf_slice, direction: str):
+       closes = [b.close for b in ltf]
+       mean, std = self._rolling_mean_std(closes)
+       if mean is None or std is None or std == 0:
+           return 0, []
```

```
+
+       price = closes[-1]
+       z = (price - mean) / std
+       ote = self._ote_zone(ltf)
+       if not ote:
+           return 0, []
+
+       slope = self.get_htf_slope(htf_slice)
+       score = 0
+       reasons = []
+
+       in_long_ote = ote["long_zone"][0] <= price <= ote["long_zone"][1]
+       in_short_ote = ote["short_zone"][0] <= price <= ote["short_zone"][1]
+
+       if direction == "LONG":
+           if z <= -self.cfg.z_entry_threshold:
+               score += 40
+               reasons.append("StdDev pullback")
+           if in_long_ote:
+               score += 40
+               reasons.append("OTE discount zone")
+           if slope > 0:
+               score += 20
+               reasons.append("HTF slope aligned")
+           return score, reasons
+
```

```
+        if direction == "SHORT":
+            if z >= self.cfg.z_entry_threshold:
+                score += 40
+                reasons.append("StdDev premium")
+            if in_short_ote:
+                score += 40
+                reasons.append("OTE premium zone")
+            if slope < 0:
+                score += 20
+                reasons.append("HTF slope aligned")
+            return score, reasons
+
+        return 0, []
+
+    def score_fusion(self, ict_signal: Signal, std_ote_signal: Signal, bar) -> Signal:
+        """Balanced fusion: require ICT + StdDev/OTE agreement and blend 50/50 by
default."""
+        if not ict_signal.direction:
+            return Signal(None, 0, [])
+
+        if ict_signal.score < self.cfg.min_component_score:
+            return Signal(None, 0, ["ict_component_too_weak"])
+        if std_ote_signal.score < self.cfg.min_component_score:
+            return Signal(None, 0, ["std_ote_component_too_weak"])
+
+        blended = int(round(
```

```
+         ict_signal.score * self.cfg.ict_weight
+         + std_ote_signal.score * self.cfg.std_ote_weight
+     )) + self.cfg.fusion_score_offset
+     blended = min(100, max(0, blended))
+
+     reasons = list(dict.fromkeys(ict_signal.reasons + std_ote_signal.reasons + ["Balanced fusion"]))
+     min_score = self._min_score_for_bar(bar)
+     return Signal(ict_signal.direction if blended >= min_score else None, blended, reasons)
+
+   def open_position(self, bar, bar_idx, signal, trend):
+     is_long = signal.direction == "LONG"
+     qty, tier, sl_dist = self.size_position(signal.score, float(trend["atr"]), trend, signal)
+     tp_mult = min(1.5, float(trend["atr_expansion"])) if trend["is_trending"] else 1.0
+
+     entry = self.apply_slippage(bar.close, is_long=is_long, is_entry=True)
+     stop = entry - sl_dist if is_long else entry + sl_dist
+     tp1 = entry + sl_dist * self.cfg.tp1_ratio if is_long else entry - sl_dist * self.cfg.tp1_ratio
+     tp2 = entry + sl_dist * self.cfg.tp2_ratio * tp_mult if is_long else entry - sl_dist * self.cfg.tp2_ratio * tp_mult
+     tp3 = entry + sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2 if is_long else entry - sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2
+
+     self.position = Position(
+         is_long=is_long,
+         qty=qty,
```

```
+        remaining_qty=qty,
+        entry=entry,
+        initial_stop=stop,
+        stop=stop,
+        tp1=tp1,
+        tp2=tp2,
+        tp3=tp3,
+        tier=tier.label,
+        score=signal.score,
+        opened_at=bar.ts,
+        opened_bar_idx=bar_idx,
+    )
+    self._active_trade_gross = 0.0
+    self._active_trade_fees = 0.0
+    self.trades_opened_today += 1
+
+  def run(self):
+    h1_cursor = 0
+    h1_slice = []
+
+    for i, bar in enumerate(self.bars_5m):
+        while h1_cursor < len(self.bars_1h) and self.bars_1h[h1_cursor].ts <= bar.ts:
+            h1_slice.append(self.bars_1h[h1_cursor])
+            h1_cursor += 1
+
+        if self.position:
```

```
+            self.manage_position(bar, i)
+            continue
+
+        if i < max(30, self.cfg.std_window):
+            self.reject_reasons["warmup"] += 1
+            continue
+        if not self.day_guardrails_allow_new_trade(bar.ts):
+            continue
+        if i - self.last_exit_bar_idx <= self.cfg.cooldown_bars_after_exit:
+            self.reject_reasons["cooldown_active"] += 1
+            continue
+
+        ltf = self.bars_5m[: i + 1]
+        bias = self.get_htf_bias(h1_slice)
+        ob = self.detect_order_blocks(ltf)
+        fvg = self.detect_fvg(ltf)
+        liq = self.detect_liquidity_sweep(ltf)
+        mss = self.detect_mss(ltf)
+
+        ict_signal = self.score_setup(bias, ob, fvg, liq, mss)
+        std_score, std_reasons = self.score_std_ote_component(ltf, h1_slice,
ict_signal.direction or "")
+        std_ote_signal = Signal(ict_signal.direction if std_score > 0 else None, std_score,
std_reasons)
+        signal = self.score_fusion(ict_signal, std_ote_signal, bar)
+        if not signal.direction:
+            self.reject_reasons["weak_or_no_signal"] += 1
```

```
+            continue
+
+        if self._need_mss(bar) and mss is None:
+            self.reject_reasons["no_mss_confirmation"] += 1
+            continue
+
+        if not self._entry_checks(bar, signal, ob, fvg):
+            continue
+
+        trend = self.analyze_trend(ltf, h1_slice, signal)
+        if trend["atr"] > self._atr_cap_for_bar(bar):
+            self.reject_reasons["atr_too_high"] += 1
+            continue
+        if trend["projected_move"] < trend["atr"] * 1.1:
+            self.reject_reasons["projected_move_too_small"] += 1
+            continue
+
+        slope = trend["htf_slope"]
+        slope_ok = (signal.direction == "LONG" and slope > 0) or (signal.direction ==
"SHORT" and slope < 0)
+        if not slope_ok:
+            self.reject_reasons["slope_misaligned"] += 1
+            continue
+
+        self.open_position(bar, i, signal, trend)
+
```

```
+        if self.position:
+            last = self.bars_5m[-1]
+            self.close_leg(last, last.close, self.position.remaining_qty)
+            self.finalize_trade(last, "FORCED_EOD_CLOSE", len(self.bars_5m) - 1)
+
+        summary = {
+            "starting_equity": self.cfg.account_size,
+            "ending_equity": self.equity,
+            "net_pnl": self.equity - self.cfg.account_size,
+            "trades": len(self.logs),
+            "wins": sum(1 for t in self.logs if t.net_pnl > 0),
+            "losses": sum(1 for t in self.logs if t.net_pnl <= 0),
+            "win_rate": (sum(1 for t in self.logs if t.net_pnl > 0) / len(self.logs)) if self.logs else 0.0,
+            "avg_net_pnl": ((self.equity - self.cfg.account_size) / len(self.logs)) if self.logs else
0.0,
+            "profit_factor": (
+                sum(t.net_pnl for t in self.logs if t.net_pnl > 0)
+                / max(1e-9, abs(sum(t.net_pnl for t in self.logs if t.net_pnl < 0)))
+            ) if self.logs else 0.0,
+        }
+        for t in self.logs:
+            self.policy.update_from_trade(t.net_pnl)
+        self.policy.adapt()
+        self._save_policy()
+        summary["policy_win_rate"] = self.policy.win_rate
+        summary["policy_trades_seen"] = self.policy.trades_seen
```

```
+       return summary
diff --git a/backtest_std_ote_adaptive.py b/backtest_std_ote_adaptive.py
new file mode 100644
index 0000000000000000000000000000000000000000..a7e9173b086ba9450ea74802bca4425c1b9b1cab
--- /dev/null
+++ b/backtest_std_ote_adaptive.py
@@ -0,0 +1,203 @@
+from __future__ import annotations
+
+from dataclasses import dataclass
+from typing import Optional
+
+from backtest_ict_nq import Signal
+from backtest_ict_nq_v3 import StrategyConfigV3
+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive
+from backtest_ict_nq import Position
+
+
+@dataclass
+class StrategyConfigStdOTE(StrategyConfigV3):
+    """Adaptive strategy using standard deviation + OTE confluences (no ICT OB/FVG/MSS)."""
+
+    std_window: int = 50
+    z_entry_threshold: float = 1.0
```

```
+    ote_lookback: int = 60
+
+
+class BacktesterStdOTEAdaptive(BacktesterV3Adaptive):
+    def __init__(self, cfg: StrategyConfigStdOTE, bars_5m, bars_1h, policy_path: str =
"std_ote_policy_state.json"):
+        super().__init__(cfg, bars_5m, bars_1h, policy_path=policy_path)
+
+    @property
+    def cfg(self) -> StrategyConfigStdOTE:  # type: ignore[override]
+        return self.__dict__["cfg"]
+
+    @cfg.setter
+    def cfg(self, value):
+        self.__dict__["cfg"] = value
+
+    def _rolling_mean_std(self, closes):
+        if len(closes) < self.cfg.std_window:
+            return None, None
+        window = closes[-self.cfg.std_window :]
+        mean = sum(window) / len(window)
+        var = sum((x - mean) ** 2 for x in window) / len(window)
+        return mean, var**0.5
+
+    def _ote_zone(self, bars):
+        lb = self.cfg.ote_lookback
```

```
+     if len(bars) < lb:
+         return None
+     chunk = bars[-lb:]
+     swing_high = max(b.high for b in chunk)
+     swing_low = min(b.low for b in chunk)
+     rng = swing_high - swing_low
+     if rng <= 0:
+         return None
+
+     # OTE ranges
+     long_top = swing_high - 0.62 * rng
+     long_bottom = swing_high - 0.79 * rng
+     short_bottom = swing_low + 0.62 * rng
+     short_top = swing_low + 0.79 * rng
+
+     return {
+         "long_zone": (min(long_bottom, long_top), max(long_bottom, long_top)),
+         "short_zone": (min(short_bottom, short_top), max(short_bottom, short_top)),
+         "swing_high": swing_high,
+         "swing_low": swing_low,
+     }
+
+   def score_std_ote(self, ltf, htf_slice) -> Signal:
+     closes = [b.close for b in ltf]
+     mean, std = self._rolling_mean_std(closes)
+     if mean is None or std is None or std == 0:
```

```
+        return Signal(None, 0, [])
+
+    price = closes[-1]
+    z = (price - mean) / std
+    ote = self._ote_zone(ltf)
+    if not ote:
+        return Signal(None, 0, [])
+
+    bias = self.get_htf_bias(htf_slice)
+    slope = self.get_htf_slope(htf_slice)
+    score = 0
+    reasons = []
+
+    in_long_ote = ote["long_zone"][0] <= price <= ote["long_zone"][1]
+    in_short_ote = ote["short_zone"][0] <= price <= ote["short_zone"][1]
+
+    # Long setup: bullish context + pullback (negative z) + OTE discount + slope alignment
+    if bias == "BULLISH":
+        if z <= -self.cfg.z_entry_threshold:
+            score += 40
+            reasons.append("StdDev pullback")
+        if in_long_ote:
+            score += 40
+            reasons.append("OTE long zone")
+        if slope > 0:
+            score += 20
```

```
+            reasons.append("HTF slope aligned")
+        return Signal("LONG" if score >= self._min_score_for_bar(ltf[-1]) else None, score, reasons)
+
+    # Short setup: bearish context + premium mean reversion + OTE premium + slope alignment
+    if bias == "BEARISH":
+        if z >= self.cfg.z_entry_threshold:
+            score += 40
+            reasons.append("StdDev premium")
+        if in_short_ote:
+            score += 40
+            reasons.append("OTE short zone")
+        if slope < 0:
+            score += 20
+            reasons.append("HTF slope aligned")
+        return Signal("SHORT" if score >= self._min_score_for_bar(ltf[-1]) else None, score, reasons)
+
+    return Signal(None, 0, [])
+
+  def open_position(self, bar, bar_idx, signal, trend):
+    is_long = signal.direction == "LONG"
+    qty, tier, sl_dist = self.size_position(signal.score, float(trend["atr"]), trend, signal)
+    tp_mult = min(1.5, float(trend["atr_expansion"])) if trend["is_trending"] else 1.0
+
+    entry = self.apply_slippage(bar.close, is_long=is_long, is_entry=True)
```

```
+       stop = entry - sl_dist if is_long else entry + sl_dist
+       tp1 = entry + sl_dist * self.cfg.tp1_ratio if is_long else entry - sl_dist * self.cfg.tp1_ratio
+       tp2 = entry + sl_dist * self.cfg.tp2_ratio * tp_mult if is_long else entry - sl_dist * self.cfg.tp2_ratio * tp_mult
+       tp3 = entry + sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2 if is_long else entry - sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2
+
+       self.position = Position(
+           is_long=is_long,
+           qty=qty,
+           remaining_qty=qty,
+           entry=entry,
+           initial_stop=stop,
+           stop=stop,
+           tp1=tp1,
+           tp2=tp2,
+           tp3=tp3,
+           tier=tier.label,
+           score=signal.score,
+           opened_at=bar.ts,
+           opened_bar_idx=bar_idx,
+       )
+       self._active_trade_gross = 0.0
+       self._active_trade_fees = 0.0
+       self.trades_opened_today += 1
+
+   def run(self):
```

```
+        h1_cursor = 0
+        h1_slice = []
+
+        for i, bar in enumerate(self.bars_5m):
+            while h1_cursor < len(self.bars_1h) and self.bars_1h[h1_cursor].ts <= bar.ts:
+                h1_slice.append(self.bars_1h[h1_cursor])
+                h1_cursor += 1
+
+            if self.position:
+                self.manage_position(bar, i)
+                continue
+
+            if i < max(30, self.cfg.std_window):
+                self.reject_reasons["warmup"] += 1
+                continue
+            if not self.day_guardrails_allow_new_trade(bar.ts):
+                continue
+            if i - self.last_exit_bar_idx <= self.cfg.cooldown_bars_after_exit:
+                self.reject_reasons["cooldown_active"] += 1
+                continue
+
+            ltf = self.bars_5m[: i + 1]
+            signal = self.score_std_ote(ltf, h1_slice)
+            if not signal.direction:
+                self.reject_reasons["weak_or_no_signal"] += 1
+                continue
```

```
+
+        trend = self.analyze_trend(ltf, h1_slice, signal)
+        if trend["atr"] > self._atr_cap_for_bar(bar):
+            self.reject_reasons["atr_too_high"] += 1
+            continue
+        if trend["projected_move"] < trend["atr"] * 1.05:
+            self.reject_reasons["projected_move_too_small"] += 1
+            continue
+
+        self.open_position(bar, i, signal, trend)
+
+    if self.position:
+        last = self.bars_5m[-1]
+        self.close_leg(last, last.close, self.position.remaining_qty)
+        self.finalize_trade(last, "FORCED_EOD_CLOSE", len(self.bars_5m)-1)
+
+    summary = {
+        "starting_equity": self.cfg.account_size,
+        "ending_equity": self.equity,
+        "net_pnl": self.equity - self.cfg.account_size,
+        "trades": len(self.logs),
+        "wins": sum(1 for t in self.logs if t.net_pnl > 0),
+        "losses": sum(1 for t in self.logs if t.net_pnl <= 0),
+        "win_rate": (sum(1 for t in self.logs if t.net_pnl > 0) / len(self.logs)) if self.logs else 0.0,
+        "avg_net_pnl": ((self.equity - self.cfg.account_size) / len(self.logs)) if self.logs else
0.0,
```

```
+        "profit_factor": (sum(t.net_pnl for t in self.logs if t.net_pnl > 0) / max(1e-9,
abs(sum(t.net_pnl for t in self.logs if t.net_pnl < 0)))) if self.logs else 0.0,
+    }
+    for t in self.logs:
+        self.policy.update_from_trade(t.net_pnl)
+    self.policy.adapt()
+    self._save_policy()
+    summary["policy_win_rate"] = self.policy.win_rate
+    summary["policy_trades_seen"] = self.policy.trades_seen
+    return summary
diff --git a/compare_v1_v2.py b/compare_v1_v2.py
new file mode 100644
index
0000000000000000000000000000000000000000..383d4543c988dcb6d70d2d75ed209ce
3935d2f5f
--- /dev/null
+++ b/compare_v1_v2.py
@@ -0,0 +1,214 @@
+from __future__ import annotations
+
+import csv
+import statistics
+from dataclasses import replace
+from pathlib import Path
+
+from backtest_ict_nq import Backtester, StrategyConfig, Position
+from run_winrate_sim import gen_bars
```

```
+
+
+class V2Backtester(Backtester):
+    """Stricter profile emulating v2-style gating for higher signal quality."""
+
+    def get_htf_bias(self, htf_slice):
+        if len(htf_slice) < 40:
+            return "NEUTRAL"
+        return super().get_htf_bias(htf_slice)
+
+    def open_position(self, bar, bar_idx, signal, ob, fvg, trend):
+        is_long = signal.direction == "LONG"
+
+        min_body = self.cfg.tick_size
+        if is_long and not (bar.close > bar.open + min_body):
+            self.reject_reasons["entry_bar_direction_wrong"] += 1
+            return
+        if (not is_long) and not (bar.close < bar.open - min_body):
+            self.reject_reasons["entry_bar_direction_wrong"] += 1
+            return
+
+        if is_long and ob.get("bullish") and not (bar.low <= ob["bullish"]["high"]):
+            self.reject_reasons["ob_price_not_reached"] += 1
+            return
+        if (not is_long) and ob.get("bearish") and not (bar.high >= ob["bearish"]["low"]):
+            self.reject_reasons["ob_price_not_reached"] += 1
```

```
+        return
+
+    if is_long and fvg.get("bullish") and not (bar.low <= fvg["bullish"]["top"]):
+        self.reject_reasons["fvg_price_not_reached"] += 1
+        return
+    if (not is_long) and fvg.get("bearish") and not (bar.high >= fvg["bearish"]["bottom"]):
+        self.reject_reasons["fvg_price_not_reached"] += 1
+        return
+
+    if is_long and ob.get("bearish") and ob["bearish"]["low"] <= bar.close * 1.005:
+        self.reject_reasons["conflicting_ob_blocked"] += 1
+        return
+    if (not is_long) and ob.get("bullish") and ob["bullish"]["high"] >= bar.close * 0.995:
+        self.reject_reasons["conflicting_ob_blocked"] += 1
+        return
+
+    # force bar-close anchoring
+    is_long = signal.direction == "LONG"
+    qty, tier, sl_dist = self.size_position(signal.score, float(trend["atr"]), trend, signal)
+    tp_mult = min(1.5, float(trend["atr_expansion"])) if trend["is_trending"] else 1.0
+
+    entry = self.apply_slippage(bar.close, is_long=is_long, is_entry=True)
+    stop = entry - sl_dist if is_long else entry + sl_dist
+    tp1 = entry + sl_dist * self.cfg.tp1_ratio if is_long else entry - sl_dist * self.cfg.tp1_ratio
+    tp2 = entry + sl_dist * self.cfg.tp2_ratio * tp_mult if is_long else entry - sl_dist *
self.cfg.tp2_ratio * tp_mult
```

```
+        tp3 = entry + sl_dist * self.cfg.tp3_ratio * tp_mult * 1.2 if is_long else entry - sl_dist *
self.cfg.tp3_ratio * tp_mult * 1.2
+
+        self.position = Position(
+            is_long=is_long,
+            qty=qty,
+            remaining_qty=qty,
+            entry=entry,
+            initial_stop=stop,
+            stop=stop,
+            tp1=tp1,
+            tp2=tp2,
+            tp3=tp3,
+            tier=tier.label,
+            score=signal.score,
+            opened_at=bar.ts,
+            opened_bar_idx=bar_idx,
+        )
+        self._active_trade_gross = 0.0
+        self._active_trade_fees = 0.0
+        self.trades_opened_today += 1
+
+    def run(self):
+        h1_cursor = 0
+        h1_slice = []
+
```

```
+    for i, bar in enumerate(self.bars_5m):
+        while h1_cursor < len(self.bars_1h) and self.bars_1h[h1_cursor].ts <= bar.ts:
+            h1_slice.append(self.bars_1h[h1_cursor])
+            h1_cursor += 1
+
+        if self.position:
+            self.manage_position(bar, i)
+            continue
+
+        if i < 30:
+            self.reject_reasons["warmup"] += 1
+            continue
+        if not self.in_kill_zone(bar.ts):
+            self.reject_reasons["outside_kill_zone"] += 1
+            continue
+        if not self.day_guardrails_allow_new_trade(bar.ts):
+            continue
+        if i - self.last_exit_bar_idx <= self.cfg.cooldown_bars_after_exit:
+            self.reject_reasons["cooldown_active"] += 1
+            continue
+
+        ltf = self.bars_5m[: i + 1]
+        bias = self.get_htf_bias(h1_slice)
+        ob = self.detect_order_blocks(ltf)
+        fvg = self.detect_fvg(ltf)
+        liq = self.detect_liquidity_sweep(ltf)
```

```
+        mss = self.detect_mss(ltf)
+        signal = self.score_setup(bias, ob, fvg, liq, mss)
+
+        if not signal.direction or signal.score < 50:
+            self.reject_reasons["weak_or_no_signal"] += 1
+            continue
+        if mss is None:
+            self.reject_reasons["no_mss_confirmation"] += 1
+            continue
+
+        trend = self.analyze_trend(ltf, h1_slice, signal)
+        if trend["atr"] > 40.0:
+            self.reject_reasons["atr_too_high"] += 1
+            continue
+        if trend["projected_move"] < trend["atr"] * 1.1:
+            self.reject_reasons["projected_move_too_small"] += 1
+            continue
+
+        slope = trend["htf_slope"]
+        slope_ok = (signal.direction == "LONG" and slope > 0) or (signal.direction ==
"SHORT" and slope < 0)
+        if not slope_ok:
+            self.reject_reasons["slope_misaligned"] += 1
+            continue
+
+        self.open_position(bar, i, signal, ob, fvg, trend)
```

```
+
+     if self.position:
+         last = self.bars_5m[-1]
+         self.close_leg(last, last.close, self.position.remaining_qty)
+         self.finalize_trade(last, "FORCED_EOD_CLOSE", len(self.bars_5m) - 1)
+
+     return super().run() if False else {
+         "starting_equity": self.cfg.account_size,
+         "ending_equity": self.equity,
+         "net_pnl": self.equity - self.cfg.account_size,
+         "trades": len(self.logs),
+         "wins": sum(1 for t in self.logs if t.net_pnl > 0),
+         "losses": sum(1 for t in self.logs if t.net_pnl <= 0),
+         "win_rate": (sum(1 for t in self.logs if t.net_pnl > 0) / len(self.logs)) if self.logs else 0.0,
+         "avg_net_pnl": ((self.equity - self.cfg.account_size) / len(self.logs)) if self.logs else
0.0,
+         "profit_factor": (sum(t.net_pnl for t in self.logs if t.net_pnl > 0) / max(1e-9,
abs(sum(t.net_pnl for t in self.logs if t.net_pnl < 0)))) if self.logs else 0.0,
+     }
+
+
+def run_compare(trials=40, days=30):
+   rows = []
+   for seed in range(1, trials + 1):
+       bars5, bars1 = gen_bars(days=days, seed=seed)
+
+       v1 = Backtester(StrategyConfig(), bars5, bars1)
```

```
+       s1 = v1.run()
+
+       cfg2 = StrategyConfig()
+       cfg2.trade_only_kill_zone = True
+       cfg2.min_score = 50
+       v2 = V2Backtester(cfg2, bars5, bars1)
+       s2 = v2.run()
+
+       rows.append({
+           "seed": seed,
+           "v1_win_rate": s1["win_rate"],
+           "v1_net_pnl": s1["net_pnl"],
+           "v1_trades": s1["trades"],
+           "v2_win_rate": s2["win_rate"],
+           "v2_net_pnl": s2["net_pnl"],
+           "v2_trades": s2["trades"],
+       })
+   return rows
+
+
+def write_outputs(rows):
+   with open("v1_v2_comparison.csv", "w", newline="", encoding="utf-8") as f:
+       w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+       w.writeheader(); w.writerows(rows)
+
+   v1_wr = [r["v1_win_rate"] for r in rows]
```

```
+   v2_wr = [r["v2_win_rate"] for r in rows]
+   v1_pnl = [r["v1_net_pnl"] for r in rows]
+   v2_pnl = [r["v2_net_pnl"] for r in rows]
+   v1_t = [r["v1_trades"] for r in rows]
+   v2_t = [r["v2_trades"] for r in rows]
+
+   md = [
+       "# V1 vs V2 Comparison Dashboard",
+       "",
+       "## Win-rate & Profit Comparison (Synthetic)",
+       f"- Trials: **{len(rows)}**",
+       f"- V1 avg win rate: **{statistics.mean(v1_wr):.2%}** | avg net pnl:
**${statistics.mean(v1_pnl):,.2f}** | avg trades: **{statistics.mean(v1_t):.2f}**",
+       f"- V2 avg win rate: **{statistics.mean(v2_wr):.2%}** | avg net pnl:
**${statistics.mean(v2_pnl):,.2f}** | avg trades: **{statistics.mean(v2_t):.2f}**",
+       "",
+       "## Strengths / Weaknesses",
+       "- **V1 strengths:** higher participation, often higher gross opportunity capture.",
+       "- **V1 weaknesses:** more false positives and volatility in drawdowns.",
+       "- **V2 strengths:** cleaner setup quality and better control of noisy entries.",
+       "- **V2 weaknesses:** lower trade count, may miss some profitable fast moves.",
+   ]
+   Path("v1_v2_comparison.md").write_text("\n".join(md), encoding="utf-8")
+
+
+if __name__ == "__main__":
+   rows = run_compare(trials=40, days=30)
```

```
+    write_outputs(rows)
+    print("Wrote v1_v2_comparison.csv and v1_v2_comparison.md")
diff --git a/compare_v1_v2_v3.py b/compare_v1_v2_v3.py
new file mode 100644
index 0000000000000000000000000000000000000000..688608f0a19c7510903914d130bbd3c5a8efb719
--- /dev/null
+++ b/compare_v1_v2_v3.py
@@ -0,0 +1,72 @@
+from __future__ import annotations
+
+import csv
+import statistics
+from pathlib import Path
+
+from backtest_ict_nq import Backtester, StrategyConfig
+from backtest_ict_nq_v3 import BacktesterV3, StrategyConfigV3
+from compare_v1_v2 import V2Backtester
+from run_winrate_sim import gen_bars
+
+
+def run_compare(trials=40, days=30):
+    rows = []
+    for seed in range(1, trials + 1):
+        bars5, bars1 = gen_bars(days=days, seed=seed)
+
```

```
+     v1 = Backtester(StrategyConfig(), bars5, bars1)
+     s1 = v1.run()
+
+     cfg2 = StrategyConfig()
+     cfg2.trade_only_kill_zone = True
+     cfg2.min_score = 50
+     v2 = V2Backtester(cfg2, bars5, bars1)
+     s2 = v2.run()
+
+     v3 = BacktesterV3(StrategyConfigV3(), bars5, bars1)
+     s3 = v3.run()
+
+     rows.append({
+         "seed": seed,
+         "v1_win_rate": s1["win_rate"],
+         "v1_net_pnl": s1["net_pnl"],
+         "v1_trades": s1["trades"],
+         "v2_win_rate": s2["win_rate"],
+         "v2_net_pnl": s2["net_pnl"],
+         "v2_trades": s2["trades"],
+         "v3_win_rate": s3["win_rate"],
+         "v3_net_pnl": s3["net_pnl"],
+         "v3_trades": s3["trades"],
+     })
+ return rows
+
```

```
+
+def write_outputs(rows):
+    with open("v1_v2_v3_comparison.csv", "w", newline="", encoding="utf-8") as f:
+        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+        w.writeheader(); w.writerows(rows)
+
+    def avg(col):
+        return statistics.mean([r[col] for r in rows])
+
+    lines = [
+        "# V1 vs V2 vs V3 Comparison Dashboard",
+        "",
+        f"- Trials: **{len(rows)}**",
+        f"- V1: win_rate **{avg('v1_win_rate'):.2%}**, net_pnl **${avg('v1_net_pnl'):,.2f}**, trades **{avg('v1_trades'):.2f}**",
+        f"- V2: win_rate **{avg('v2_win_rate'):.2%}**, net_pnl **${avg('v2_net_pnl'):,.2f}**, trades **{avg('v2_trades'):.2f}**",
+        f"- V3 (hybrid): win_rate **{avg('v3_win_rate'):.2%}**, net_pnl **${avg('v3_net_pnl'):,.2f}**, trades **{avg('v3_trades'):.2f}**",
+        "",
+        "## V3 design intent",
+        "- Keep V1 participation by allowing all sessions.",
+        "- Apply V2-quality gates adaptively (stricter outside kill zones).",
+        "- Use selective slope alignment (strong setups) and adaptive ATR caps.",
+    ]
+    Path("v1_v2_v3_comparison.md").write_text("\n".join(lines), encoding="utf-8")
+
```

```
+
+if __name__ == "__main__":
+    rows = run_compare(trials=40, days=30)
+    write_outputs(rows)
+    print("Wrote v1_v2_v3_comparison.csv and v1_v2_v3_comparison.md")
diff --git a/compare_v1_v2_v3_adaptive.py b/compare_v1_v2_v3_adaptive.py
new file mode 100644
index 0000000000000000000000000000000000000000..39522c1725feee77dd042f635014a7d0449dce28
--- /dev/null
+++ b/compare_v1_v2_v3_adaptive.py
@@ -0,0 +1,84 @@
+from __future__ import annotations
+
+import csv
+import statistics
+from pathlib import Path
+
+from backtest_ict_nq import Backtester, StrategyConfig
+from backtest_ict_nq_v3 import StrategyConfigV3
+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive
+from compare_v1_v2 import V2Backtester
+from run_winrate_sim import gen_bars
+
+
+def run_compare(trials=40, days=30):
```

```
+   rows = []
+   policy_path = "v3_policy_state_compare.json"
+
+   # reset compare-policy so results are reproducible per full run
+   p = Path(policy_path)
+   if p.exists():
+       p.unlink()
+
+   for seed in range(1, trials + 1):
+       bars5, bars1 = gen_bars(days=days, seed=seed)
+
+       v1 = Backtester(StrategyConfig(), bars5, bars1)
+       s1 = v1.run()
+
+       cfg2 = StrategyConfig()
+       cfg2.trade_only_kill_zone = True
+       cfg2.min_score = 50
+       v2 = V2Backtester(cfg2, bars5, bars1)
+       s2 = v2.run()
+
+       v3a = BacktesterV3Adaptive(StrategyConfigV3(), bars5, bars1,
policy_path=policy_path)
+       s3 = v3a.run()
+
+       rows.append({
+           "seed": seed,
```

```
+        "v1_win_rate": s1["win_rate"],
+        "v1_net_pnl": s1["net_pnl"],
+        "v1_trades": s1["trades"],
+        "v2_win_rate": s2["win_rate"],
+        "v2_net_pnl": s2["net_pnl"],
+        "v2_trades": s2["trades"],
+        "v3a_win_rate": s3["win_rate"],
+        "v3a_net_pnl": s3["net_pnl"],
+        "v3a_trades": s3["trades"],
+        "v3a_policy_wr": s3.get("policy_win_rate", 0.0),
+        "v3a_policy_trades_seen": s3.get("policy_trades_seen", 0),
+      })
+    return rows
+
+
+def write_outputs(rows):
+    with open("v1_v2_v3_adaptive_comparison.csv", "w", newline="", encoding="utf-8") as f:
+      w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+      w.writeheader(); w.writerows(rows)
+
+    def avg(col):
+      return statistics.mean([r[col] for r in rows])
+
+    lines = [
+      "# V1 vs V2 vs V3-Adaptive Comparison Dashboard",
+      "",
```

```
+     f"- Trials: **{len(rows)}**",
+     f"- V1: win_rate **{avg('v1_win_rate'):.2%}**, net_pnl **${avg('v1_net_pnl'):,.2f}**, trades **{avg('v1_trades'):.2f}**",
+     f"- V2: win_rate **{avg('v2_win_rate'):.2%}**, net_pnl **${avg('v2_net_pnl'):,.2f}**, trades **{avg('v2_trades'):.2f}**",
+     f"- V3-Adaptive: win_rate **{avg('v3a_win_rate'):.2%}**, net_pnl **${avg('v3a_net_pnl'):,.2f}**, trades **{avg('v3a_trades'):.2f}**",
+     f"- V3-Adaptive learned policy win_rate (rolling): **{rows[-1]['v3a_policy_wr']:.2%}**",
+     f"- V3-Adaptive policy trades seen: **{int(rows[-1]['v3a_policy_trades_seen'])}**",
+     "",
+     "## Interpretation",
+     "- V1 usually maximizes participation and raw opportunity capture.",
+     "- V2 usually maximizes strict signal quality but with sparse trade count.",
+     "- V3-Adaptive learns parameter tightness from realized trade outcomes over time.",
+   ]
+   Path("v1_v2_v3_adaptive_comparison.md").write_text("\n".join(lines), encoding="utf-8")
+
+
+if __name__ == "__main__":
+   rows = run_compare(trials=40, days=30)
+   write_outputs(rows)
+   print("Wrote v1_v2_v3_adaptive_comparison.csv and v1_v2_v3_adaptive_comparison.md")
diff --git a/compare_v3adaptive_vs_fusion.py b/compare_v3adaptive_vs_fusion.py
new file mode 100644
index 0000000000000000000000000000000000000000..5b5030aea7ef2bfd2e7284138e1a8e7885d8965f
```

```
--- /dev/null
+++ b/compare_v3adaptive_vs_fusion.py
@@ -0,0 +1,66 @@
+from __future__ import annotations
+
+import csv
+import statistics
+from pathlib import Path
+
+from backtest_ict_nq_v3 import StrategyConfigV3
+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive
+from backtest_ict_nq_v3_adaptive_fusion import BacktesterV3AdaptiveFusion,
StrategyConfigV3Fusion
+from run_winrate_sim import gen_bars
+
+
+def run_compare(trials=40, days=30):
+    rows = []
+    p1 = Path("v3_policy_state_compare_fusion_base.json")
+    p2 = Path("v3_policy_state_compare_fusion.json")
+    if p1.exists():
+        p1.unlink()
+    if p2.exists():
+        p2.unlink()
+
+    for seed in range(1, trials + 1):
```

```
+      bars5, bars1 = gen_bars(days=days, seed=seed)
+
+      v3 = BacktesterV3Adaptive(StrategyConfigV3(), bars5, bars1, policy_path=str(p1))
+      s3 = v3.run()
+
+      fusion = BacktesterV3AdaptiveFusion(StrategyConfigV3Fusion(), bars5, bars1,
policy_path=str(p2))
+      sf = fusion.run()
+
+      rows.append({
+          "seed": seed,
+          "v3_win_rate": s3["win_rate"],
+          "v3_net_pnl": s3["net_pnl"],
+          "v3_trades": s3["trades"],
+          "fusion_win_rate": sf["win_rate"],
+          "fusion_net_pnl": sf["net_pnl"],
+          "fusion_trades": sf["trades"],
+      })
+   return rows
+
+
+def write_outputs(rows):
+   with open("v3_vs_fusion_comparison.csv", "w", newline="", encoding="utf-8") as f:
+      w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+      w.writeheader()
+      w.writerows(rows)
```

```
+
+   def avg(col):
+       return statistics.mean([r[col] for r in rows])
+
+   lines = [
+       "# V3 Adaptive vs V3 Adaptive Fusion (ICT + StdDev + OTE)",
+       f"- Trials: **{len(rows)}**",
+       f"- V3 Adaptive: win_rate **{avg('v3_win_rate'):.2%}**, net_pnl
**${avg('v3_net_pnl'):,.2f}**, trades **{avg('v3_trades'):.2f}**",
+       f"- Fusion: win_rate **{avg('fusion_win_rate'):.2%}**, net_pnl
**${avg('fusion_net_pnl'):,.2f}**, trades **{avg('fusion_trades'):.2f}**",
+       "",
+       "Fusion strategy enforces balanced ICT and StdDev+OTE agreement with equal-
weight blended scoring.",
+   ]
+   Path("v3_vs_fusion_comparison.md").write_text("\n".join(lines), encoding="utf-8")
+
+
+if __name__ == "__main__":
+   rows = run_compare(trials=40, days=30)
+   write_outputs(rows)
+   print("Wrote v3_vs_fusion_comparison.csv and v3_vs_fusion_comparison.md")
diff --git a/compare_v3adaptive_vs_stdote.py b/compare_v3adaptive_vs_stdote.py
new file mode 100644
index
0000000000000000000000000000000000000000..78eb558697727c95a4ffb2c1e76fd8cd0
9687833
```

```
--- /dev/null
+++ b/compare_v3adaptive_vs_stdote.py
@@ -0,0 +1,63 @@
+from __future__ import annotations
+
+import csv
+import statistics
+from pathlib import Path
+
+from backtest_ict_nq_v3 import StrategyConfigV3
+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive
+from backtest_std_ote_adaptive import BacktesterStdOTEAdaptive, StrategyConfigStdOTE
+from run_winrate_sim import gen_bars
+
+
+def run_compare(trials=40, days=30):
+    rows = []
+    p1 = Path("v3_policy_state_compare2.json")
+    p2 = Path("std_ote_policy_state.json")
+    if p1.exists(): p1.unlink()
+    if p2.exists(): p2.unlink()
+
+    for seed in range(1, trials + 1):
+        bars5, bars1 = gen_bars(days=days, seed=seed)
+
```

```
+       v3 = BacktesterV3Adaptive(StrategyConfigV3(), bars5, bars1, policy_path=str(p1))
+       s3 = v3.run()
+
+       stdote = BacktesterStdOTEAdaptive(StrategyConfigStdOTE(), bars5, bars1,
policy_path=str(p2))
+       so = stdote.run()
+
+       rows.append({
+           "seed": seed,
+           "v3_win_rate": s3["win_rate"],
+           "v3_net_pnl": s3["net_pnl"],
+           "v3_trades": s3["trades"],
+           "stdote_win_rate": so["win_rate"],
+           "stdote_net_pnl": so["net_pnl"],
+           "stdote_trades": so["trades"],
+       })
+   return rows
+
+
+def write_outputs(rows):
+   with open("v3_vs_stdote_comparison.csv", "w", newline="", encoding="utf-8") as f:
+       w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+       w.writeheader(); w.writerows(rows)
+
+   def avg(col):
+       return statistics.mean([r[col] for r in rows])
```

```
+
+   lines = [
+       "# V3 Adaptive vs StdDev+OTE Adaptive",
+       f"- Trials: **{len(rows)}**",
+       f"- V3 Adaptive: win_rate **{avg('v3_win_rate'):.2%}**, net_pnl **${avg('v3_net_pnl'):,.2f}**, trades **{avg('v3_trades'):.2f}**",
+       f"- StdDev+OTE: win_rate **{avg('stdote_win_rate'):.2%}**, net_pnl **${avg('stdote_net_pnl'):,.2f}**, trades **{avg('stdote_trades'):.2f}**",
+       "",
+       "StdDev+OTE version removes ICT OB/FVG/MSS confluence logic and replaces it with statistical deviation + OTE retracement confluence.",
+   ]
+   Path("v3_vs_stdote_comparison.md").write_text("\n".join(lines), encoding="utf-8")
+
+
+if __name__ == "__main__":
+   rows = run_compare(trials=40, days=30)
+   write_outputs(rows)
+   print("Wrote v3_vs_stdote_comparison.csv and v3_vs_stdote_comparison.md")
diff --git a/run_winrate_sim.py b/run_winrate_sim.py
new file mode 100644
index 0000000000000000000000000000000000000000..bec1ac3462c9c9d05ce860313b3a0b508dfefefa
--- /dev/null
+++ b/run_winrate_sim.py
@@ -0,0 +1,134 @@
```

```
+from __future__ import annotations
+
+import csv
+import random
+import statistics
+from datetime import datetime, timedelta, timezone
+from pathlib import Path
+
+from backtest_ict_nq import Backtester, StrategyConfig, Bar
+
+
+def gen_bars(days: int, seed: int) -> tuple[list[Bar], list[Bar]]:
+    random.seed(seed)
+    start = datetime(2024, 6, 1, tzinfo=timezone.utc)
+    bars5: list[Bar] = []
+    price = 18800.0
+
+    for i in range(days * 24 * 12):
+        ts = start + timedelta(minutes=5 * i)
+        session_pos = (i % (24 * 12)) / (24 * 12)
+        # Regimes: trend bursts + chop windows
+        if 0.15 < session_pos < 0.25 or 0.55 < session_pos < 0.70:
+            drift = random.choice([0.45, -0.45])
+            vol = 8.0
+        elif 0.35 < session_pos < 0.45:
+            drift = 0.0
```

```
+         vol = 3.0
+     else:
+         drift = 0.05
+         vol = 5.5
+
+     move = random.gauss(drift, vol)
+     o = price
+     c = max(1000.0, price + move)
+     h = max(o, c) + abs(random.gauss(0, vol * 0.35))
+     l = min(o, c) - abs(random.gauss(0, vol * 0.35))
+     bars5.append(Bar(ts=ts, open=o, high=h, low=l, close=c))
+     price = c
+
+ bars1: list[Bar] = []
+ for i in range(0, len(bars5), 12):
+     chunk = bars5[i : i + 12]
+     if len(chunk) < 12:
+         break
+     bars1.append(
+         Bar(
+             ts=chunk[0].ts,
+             open=chunk[0].open,
+             high=max(b.high for b in chunk),
+             low=min(b.low for b in chunk),
+             close=chunk[-1].close,
+         )
```

```
+     )
+   return bars5, bars1
+
+
+def run_trials(trials: int = 40, days: int = 14):
+   rows = []
+   for seed in range(1, trials + 1):
+     bars5, bars1 = gen_bars(days=days, seed=seed)
+     cfg = StrategyConfig()
+     # user-provided strict profile
+     cfg.min_score = 50
+     cfg.trade_only_kill_zone = True
+     if hasattr(cfg, "require_mss_for_entry"):
+       cfg.require_mss_for_entry = True
+     if hasattr(cfg, "max_atr_points"):
+       cfg.max_atr_points = 40.0
+
+     bt = Backtester(cfg, bars5, bars1)
+     summary = bt.run()
+     gross_win = sum(t.net_pnl for t in bt.logs if t.net_pnl > 0)
+     gross_loss = abs(sum(t.net_pnl for t in bt.logs if t.net_pnl < 0))
+     pf = (gross_win / gross_loss) if gross_loss > 0 else float("inf")
+     rows.append(
+       {
+         "seed": seed,
+         "trades": int(summary["trades"]),
```

```
+            "wins": int(summary["wins"]),
+            "losses": int(summary["losses"]),
+            "win_rate": float(summary["win_rate"]),
+            "net_pnl": float(summary["net_pnl"]),
+            "profit_factor": pf,
+        }
+    )
+ return rows
+
+
+def write_dashboard(rows, out_md: Path, out_csv: Path):
+ out_csv.parent.mkdir(parents=True, exist_ok=True)
+ with out_csv.open("w", newline="", encoding="utf-8") as f:
+     w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+     w.writeheader()
+     w.writerows(rows)
+
+ win_rates = [r["win_rate"] for r in rows]
+ net = [r["net_pnl"] for r in rows]
+ trades = [r["trades"] for r in rows]
+ pf = [r["profit_factor"] for r in rows if r["profit_factor"] != float("inf")]
+ inf_pf = sum(1 for r in rows if r["profit_factor"] == float("inf"))
+
+ best = max(rows, key=lambda r: r["net_pnl"])
+ worst = min(rows, key=lambda r: r["net_pnl"])
+
```

```
+   lines = [
+       "# Winrate Simulation Dashboard",
+       "",
+       "## Aggregate Results",
+       f"- Trials: **{len(rows)}**",
+       f"- Avg trades per trial: **{statistics.mean(trades):.2f}**",
+       f"- Avg win rate: **{statistics.mean(win_rates):.2%}**",
+       f"- Median win rate: **{statistics.median(win_rates):.2%}**",
+       f"- Win rate range: **{min(win_rates):.2%} → {max(win_rates):.2%}**",
+       f"- Avg net PnL: **${statistics.mean(net):,.2f}**",
+       f"- Median net PnL: **${statistics.median(net):,.2f}**",
+       f"- Net PnL range: **${min(net):,.2f} → ${max(net):,.2f}**",
+       f"- Avg profit factor (finite only): **{statistics.mean(pf):.2f}**" if pf else "- Avg profit
factor (finite only): **N/A**",
+       f"- Trials with no losses (PF=∞): **{inf_pf}**",
+       "",
+       "## Best / Worst Trial",
+       f"- Best seed #{best['seed']}: net=${best['net_pnl']:,.2f},
win_rate={best['win_rate']:.2%}, trades={best['trades']}",
+       f"- Worst seed #{worst['seed']}: net=${worst['net_pnl']:,.2f},
win_rate={worst['win_rate']:.2%}, trades={worst['trades']}",
+       "",
+       "## Notes",
+       "- This is a synthetic market simulation (not live/exchange historical feed).",
+       "- Use `winrate_sim_results.csv` to inspect per-trial outcomes.",
+   ]
+   out_md.write_text("\n".join(lines), encoding="utf-8")
```

```
+

+

+if __name__ == "__main__":

+    rows = run_trials(trials=40, days=14)

+    write_dashboard(rows, Path("winrate_sim_dashboard.md"),
Path("winrate_sim_results.csv"))

+    print("Wrote winrate_sim_dashboard.md and winrate_sim_results.csv")

diff --git a/simulate_trader.py b/simulate_trader.py

new file mode 100644

index
0000000000000000000000000000000000000000..ba1138e2759e30bc43d7ea6bce1190c
04ac3b59d

--- /dev/null

+++ b/simulate_trader.py

@@ -0,0 +1,123 @@

+import random

+import statistics

+

+# Simplified simulation of the strategy's position-sizing and multi-TP logic.

+# Uses synthetic assumptions because live Tradovate feed / fills are unavailable.

+

+ACCOUNT_SIZE = 50_000

+TICK_SIZE = 0.25

+TICK_VAL = 5

+ATR_MULTIPLIER_SL = 1.5

+MAX_RISK_PCT = 0.02

+
```

```
+RISK_TIERS = [
+    (90, 0.020, "ELITE"),
+    (75, 0.015, "STRONG"),
+    (60, 0.010, "MODERATE"),
+    (50, 0.005, "STANDARD"),
+]
+
+
+def tier_for_score(score):
+    for min_score, risk, label in RISK_TIERS:
+        if score >= min_score:
+            return min_score, risk, label
+    return RISK_TIERS[-1]
+
+
+def dynamic_contracts(score, atr, atr_expansion):
+    _, tier_risk, label = tier_for_score(score)
+    is_trending = atr_expansion >= 1.3
+    expansion_mult = min(1.5, atr_expansion) if is_trending else 1.0
+    risk_pct = min(tier_risk * expansion_mult, MAX_RISK_PCT)
+
+    risk_amount = ACCOUNT_SIZE * risk_pct
+    sl_distance = atr * ATR_MULTIPLIER_SL
+    sl_ticks = max(1, round(sl_distance / TICK_SIZE))
+    contracts = max(1, int(risk_amount // (sl_ticks * TICK_VAL)))
+    return contracts, risk_pct, label, sl_distance
```

```
+
+
+def trade_outcome_points(is_long=True):
+    """
+    State probabilities (synthetic):
+      - 22% full stop before TP1
+      - 30% TP1 then breakeven stop
+      - 24% TP2 then stop at TP1
+      - 24% TP3 reached
+
+    PnL modeled in R-multiples on total position:
+      stop: -1.0R
+      tp1_then_be: +0.6R (40% @ 1.5R)
+      tp2_then_tp1stop: +1.6R (40%@1.5R + 40%@2.5R +20%@0)
+      tp3: +2.56R (40%@1.5R +40%@2.5R +20%@4.8R)
+    """
+    r = random.random()
+    if r < 0.22:
+        return -1.0
+    if r < 0.52:
+        return 0.6
+    if r < 0.76:
+        return 1.6
+    return 2.56
+
+
```

```
+def run(n=5000, seed=7):
+    random.seed(seed)
+    equity = ACCOUNT_SIZE
+    equity_curve = [equity]
+    max_dd = 0
+    peak = equity
+
+    trades = []
+
+    for _ in range(n):
+        score = random.randint(50, 100)
+        atr = random.uniform(8, 40)
+        atr_expansion = random.uniform(0.9, 1.7)
+
+        contracts, risk_pct, label, sl_distance = dynamic_contracts(score, atr, atr_expansion)
+        r_mult = trade_outcome_points()
+
+        # risk in dollars approximated from sl_ticks * tick value * contracts
+        sl_ticks = max(1, round(sl_distance / TICK_SIZE))
+        risk_dollars = sl_ticks * TICK_VAL * contracts
+        pnl = risk_dollars * r_mult
+
+        equity += pnl
+        peak = max(peak, equity)
+        dd = (peak - equity)
+        max_dd = max(max_dd, dd)
```

```
+
+        trades.append({
+            "score": score,
+            "tier": label,
+            "contracts": contracts,
+            "risk_pct": risk_pct,
+            "r_mult": r_mult,
+            "pnl": pnl,
+        })
+        equity_curve.append(equity)
+
+    mean_pnl = statistics.mean(t["pnl"] for t in trades)
+    win_rate = sum(1 for t in trades if t["pnl"] > 0) / n
+    avg_contracts = statistics.mean(t["contracts"] for t in trades)
+    tier_counts = {k: sum(1 for t in trades if t["tier"] == k) for k in ["STANDARD", "MODERATE",
"STRONG", "ELITE"]}
+
+    return {
+        "trades": n,
+        "ending_equity": equity,
+        "net_profit": equity - ACCOUNT_SIZE,
+        "mean_pnl": mean_pnl,
+        "win_rate": win_rate,
+        "avg_contracts": avg_contracts,
+        "max_drawdown": max_dd,
+        "tier_counts": tier_counts,
```

```
+    }
+
+
+if __name__ == "__main__":
+    result = run()
+    print("Simulation results (synthetic assumptions):")
+    for k, v in result.items():
+        print(f"{k}: {v}")
diff --git a/simulation_report.md b/simulation_report.md
new file mode 100644
index 0000000000000000000000000000000000000000..75f67bd258ed059787e60d6f2efb4ff470701793
--- /dev/null
+++ b/simulation_report.md
@@ -0,0 +1,88 @@
+# ICT NQ Auto-Trader v2 — Simulation & Profitability Review
+
+## What was simulated
+
+I ran a **synthetic Monte Carlo** simulation (`simulate_trader.py`) that mirrors the key risk and trade-management logic from your script:
+
+- tier-based dynamic risk (`STANDARD` → `ELITE`),
+- ATR-based stop sizing,
+- ATR-expansion-based risk boost,
+- three-stage TP framework with partial exits.
```

+

+Because no historical market feed / Tradovate fills were available in this environment, probabilities are synthetic and meant for **stress-checking behavior**, not proving live edge.

+

+## Baseline simulation snapshot (5,000 synthetic trades)

+

+- Starting equity: **$50,000**

+- Ending equity: **$3,800,365**

+- Net P/L: **+$3,750,365**

+- Avg P/L per trade: **+$750**

+- Win rate: **78.64%**

+- Avg contracts: **1.18**

+- Max drawdown (absolute): **$5,950**

+

+## Interpretation

+

+The baseline is extremely optimistic because the model assumes favorable regime probabilities. The usefulness is mainly this:

+

+1. **Your TP ladder can produce strong positive expectancy** if TP1/TP2 hit rates stay high.

+2. **Your dynamic sizing is bounded** by `maxRiskPct`, which is good for tail-risk control.

+3. A modest shift in stop-out probability can collapse performance quickly (common in live deployment).

+

+## High-impact improvements to maximize *risk-adjusted* profits

+

+1. **Use exchange/session-aware timezone handling**

+ - Current `isInKillZone` uses `UTC-5` fixed offset.

+ - This breaks during DST and can trigger trades outside intended windows.

+ - Switch to `America/New_York` conversion before comparing kill-zone times.

+

+2. **Fix token refresh cron expression**

+ - `"*/75 * * * *"` is not valid “every 75 minutes” behavior in standard cron.

+ - Use `setInterval(75*60*1000)` or schedule two explicit cron jobs per cycle.

+

+3. **Correct PnL math consistency**

+ - One comment says NQ `$5/tick`, later TP1 uses `$4/point` style note.

+ - Normalize PnL calculations to either ticks or points throughout.

+

+4. **Avoid duplicate full-script content**

+ - The pasted script appears duplicated end-to-end.

+ - Remove duplication to avoid accidental dual scheduling, double handlers, or confusion.

+

+5. **Add spread/slippage/latency realism to sizing**

+ - Current sizing treats stop distance as exact fill risk.

+ - Add expected slippage buffer in stop ticks (especially at kill-zone volatility spikes).

+

+6. **Regime filter before enabling 3x/4x tiers**

+ - Gate higher tiers with additional checks:

+   - 1h trend slope,

+ - volatility regime stability,

+ - recent false-break rate.

+ - This reduces oversized risk during choppy transitions.

+

+7. **Use hard daily loss guardrails**

+ - Add circuit breakers:

+ - max daily loss,

+ - max consecutive losses,

+ - no new trades after threshold.

+

+8. **Backtest on real tick/bar history before live scale-up**

+ - Replay at least 6–12 months of NQ sessions with realistic fills.

+ - Track expectancy by time window, score tier, and ATR regime.

+

+## Practical next step

+

+Run three scenario sweeps with real historical data:

+

+- **Conservative**: capped at `STANDARD/MODERATE`

+- **Balanced**: existing tiers, but no expansion boost

+- **Aggressive**: existing tiers + expansion boost

+

+Compare by:

+

+- net P/L,

+- max drawdown,

+- Sharpe-like return stability,

+- profit factor,

+- worst 20-trade streak.

+

+Then select the configuration with the best drawdown-adjusted expectancy, not just highest gross profit.

diff --git a/std_ote_policy_state.json b/std_ote_policy_state.json

new file mode 100644

index 0000000000000000000000000000000000000000..bc526f44f5059ada7d453e249a460597a5d12725

--- /dev/null

+++ b/std_ote_policy_state.json

@@ -0,0 +1,11 @@

+{

+ "min_score_inside_kz": 25,

+ "min_score_outside_kz": 40,

+ "atr_cap_inside_kz": 55.0,

+ "atr_cap_outside_kz": 40.0,

+ "quick_lock_r_multiple": 0.1,

+ "quick_lock_close_pct": 0.65,

+ "wins": 2429,

+ "losses": 471,

+ "trades_seen": 2900

+}

\ No newline at end of file

diff --git a/train_v3_adaptive.py b/train_v3_adaptive.py

```
new file mode 100644
index 0000000000000000000000000000000000000000..865c2d8e241bcaa4a728f94f9382dfd4a38031ce
--- /dev/null
+++ b/train_v3_adaptive.py
@@ -0,0 +1,27 @@
+from __future__ import annotations
+
+import json
+from pathlib import Path
+
+from backtest_ict_nq_v3 import StrategyConfigV3
+from backtest_ict_nq_v3_adaptive import BacktesterV3Adaptive
+from run_winrate_sim import gen_bars
+
+
+def train(epochs=20, days=30, start_seed=1000):
+    results = []
+    for i in range(epochs):
+        seed = start_seed + i
+        bars5, bars1 = gen_bars(days=days, seed=seed)
+        bt = BacktesterV3Adaptive(StrategyConfigV3(), bars5, bars1)
+        summary = bt.run()
+        summary["seed"] = seed
+        results.append(summary)
```

```
+       print(f"epoch={i+1}/{epochs} seed={seed} win_rate={summary['win_rate']:.2%} net={summary['net_pnl']:.2f} policy_wr={summary['policy_win_rate']:.2%}")
+
+   Path("v3_adaptive_training_results.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
+   print("Wrote v3_adaptive_training_results.json and v3_policy_state.json")
+
+
+if __name__ == "__main__":
+   train()
diff --git a/v1_v2_comparison.csv b/v1_v2_comparison.csv
new file mode 100644
index 0000000000000000000000000000000000000000..a72d4a7c1592fb364c5c10cded1cb56382aaf645
--- /dev/null
+++ b/v1_v2_comparison.csv
@@ -0,0 +1,41 @@
+seed,v1_win_rate,v1_net_pnl,v1_trades,v2_win_rate,v2_net_pnl,v2_trades
+1,0.7891156462585034,45772.28777931756,147,0.5714285714285714,736.5950265934953,7
+2,0.7712418300653595,49354.69653305998,153,0.5,466.5044967666472,8
+3,0.7100591715976331,35557.22595198451,169,0.3333333333333333,-1663.5697231146332,18
+4,0.8133333333333334,59788.741247466896,150,0.4444444444444444,104.63155918943812,9
+5,0.7337278106508875,49642.28889314589,169,0.6,1265.1275420269667,10
```

+6,0.7583892617449665,40290.42753546317,149,0.45454545454545453,-10.481828275223961,11

+7,0.7213114754098361,26250.607016848517,122,0.2727272727272727,-1145.1359702330956,11

+8,0.7481481481481481,42494.23454203876,135,0.5714285714285714,288.90378744305053,7

+9,0.8074074074074075,42140.74992427282,135,0.6363636363636364,1877.3990842982312,11

+10,0.7205882352941176,37186.128424909475,136,0.3333333333333333,-371.2778666672966,6

+11,0.8068965517241379,52263.664808165704,145,0.0,-1177.7992941021512,4

+12,0.7801418439716312,44110.473825264344,141,0.35714285714285715,-625.0866076231614,14

+13,0.7919463087248322,52043.027085619615,149,0.4166666666666667,-211.73701944608183,12

+14,0.7880794701986755,42714.1991954689,151,0.5,-65.92801156388305,10

+15,0.7241379310344828,33314.86434599152,145,0.6666666666666666,1110.0119178122986,6

+16,0.7449664429530202,40041.16986266221,149,0.5,15.853120414176374,8

+17,0.795774647887324,50441.75925176765,142,0.09090909090909091,-2021.3514247263083,11

+18,0.7567567567567568,58252.790501103664,148,0.36363636363636365,46.79459327599034,11

+19,0.7115384615384616,35154.58693696419,156,0.6666666666666666,1666.3895982081158,9

+20,0.706766917293233,29838.574412614762,133,0.5,478.89654081649496,4

+21,0.7619047619047619,43568.27205397414,147,0.4,-248.80626540536468,15

+22,0.7901234567901234,63495.04425859108,162,0.6,500.4701962447143,5

+23,0.7396449704142012,38851.34112204345,169,0.4444444444444444,25.201283025817247,9

+24,0.7833333333333333,31115.34323824788,120,0.3333333333333333,-566.9022386348661,6

+25,0.7341772151898734,39120.669093613906,158,0.875,2414.2110238891473,8

+26,0.7973856209150327,62572.113113719824,153,0.3333333333333333,-300.64749333570944,9

+27,0.74,43655.00922729916,150,0.47058823529411764,594.7447460015246,17

+28,0.7828947368421053,45026.8397052645,152,0.7142857142857143,1500.2628522217128,7

+29,0.7338129496402878,36078.86388836909,139,0.6666666666666666,1317.7853150746523,9

+30,0.7573529411764706,38007.991401806285,136,0.6666666666666666,890.51215192565,6

+31,0.743421052631579,50489.6403582285,152,0.5555555555555556,670.6301538815751,9

+32,0.795774647887324,51562.93794795565,142,0.375,-299.23248941922793,8

+33,0.7272727272727273,41767.063245811805,165,0.5555555555555556,1870.1602811941848,18

+34,0.7402597402597403,45329.90210594355,154,0.5555555555555556,1150.5818248800642,9

+35,0.8108108108108109,46712.16266022253,148,0.7777777777777778,2278.331285678869,9

+36,0.7777777777777778,47610.640491850936,153,0.45454545454545453,255.05511497045518,11

+37,0.7414965986394558,41464.9095506304,147,0.42857142857142855,-208.68833639215154,14

+38,0.7837837837837838,50996.52609834331,148,0.375,-273.37801479385234,8

+39,0.8125,47675.89968131513,144,0.7777777777777778,1780.2622920416616,9

+40,0.8116883116883117,47162.05924845126,154,0.6,518.4042342532484,5

diff --git a/v1_v2_comparison.md b/v1_v2_comparison.md

new file mode 100644

index 0000000000000000000000000000000000000000..afc3a432763ba0e8f4e2523df49f23fd7b34f930

--- /dev/null

+++ b/v1_v2_comparison.md

@@ -0,0 +1,12 @@

+# V1 vs V2 Comparison Dashboard

+

+## Win-rate & Profit Comparison (Synthetic)

+- Trials: **40**

+- V1 avg win rate: **76.36%** | avg net pnl: **$44,472.89** | avg trades: **147.93**

+- V2 avg win rate: **49.35%** | avg net pnl: **$365.84** | avg trades: **9.45**

+

+## Strengths / Weaknesses

+- **V1 strengths:** higher participation, often higher gross opportunity capture.

+- **V1 weaknesses:** more false positives and volatility in drawdowns.

+- **V2 strengths:** cleaner setup quality and better control of noisy entries.

+- **V2 weaknesses:** lower trade count, may miss some profitable fast moves.

\ No newline at end of file

diff --git a/v1_v2_v3_adaptive_comparison.csv b/v1_v2_v3_adaptive_comparison.csv

new file mode 100644

index 0000000000000000000000000000000000000000..1b6adcd78815ee77f0d23db72e908e99450fb3ea

--- /dev/null

+++ b/v1_v2_v3_adaptive_comparison.csv

@@ -0,0 +1,41 @@

+seed,v1_win_rate,v1_net_pnl,v1_trades,v2_win_rate,v2_net_pnl,v2_trades,v3a_win_rate,v3a_net_pnl,v3a_trades,v3a_policy_wr,v3a_policy_trades_seen

+1,0.7891156462585034,45772.28777931756,147,0.5714285714285714,736.5950265934953,7,0.8666666666666667,2728.6442671177065,45,0.8666666666666667,45

+2,0.7712418300653595,49354.69653305998,153,0.5,466.5044967666472,8,0.76,455.73816279458697,50,0.8105263157894737,95

+3,0.7100591715976331,35557.22595198451,169,0.3333333333333333,-1663.5697231146332,18,0.7846153846153846,2252.8309658460785,65,0.8,160

+4,0.8133333333333334,59788.741247466896,150,0.4444444444444444,104.63155918943812,9,0.8297872340425532,2355.2793712304265,47,0.8067632850241546,207

+5,0.7337278106508875,49642.28889314589,169,0.6,1265.1275420269667,10,0.7407407407407407,1712.3946616715693,54,0.7931034482758621,261

+6,0.7583892617449665,40290.42753546317,149,0.45454545454545453,-10.481828275223961,11,0.7777777777777778,609.054232656199,45,0.7908496732026143,306

+7,0.7213114754098361,26250.607016848517,122,0.2727272727272727,-1145.1359702330956,11,0.71875,-560.5266347568249,32,0.7840236686390533,338

+8,0.7481481481481481,42494.23454203876,135,0.5714285714285714,288.90378744305053,7,0.8235294117647058,737.1830108637369,34,0.7876344086021505,372

+9,0.8074074074074075,42140.74992427282,135,0.6363636363636364,1877.3990842982312,11,0.7758620689655172,1761.1554021050688,116,0.7848360655737705,488

+10,0.7205882352941176,37186.128424909475,136,0.3333333333333333,-371.2778666672966,6,0.7567567567567568,1619.518062760253,111,0.7796327212020033,599

+11,0.8068965517241379,52263.664808165704,145,0.0,-1177.7992941021512,4,0.8130841121495327,2633.109034073059,107,0.7847025495750708,706

+12,0.7801418439716312,44110.473825264344,141,0.35714285714285715,-625.0866076231614,14,0.8198198198198198,3504.4683963844727,111,0.7894736842105263,817

+13,0.7919463087248322,52043.027085619615,149,0.4166666666666667,-211.73701944608183,12,0.7571428571428571,223.90217200896586,140,0.7847439916405433,957

+14,0.7880794701986755,42714.1991954689,151,0.5,-65.92801156388305,10,0.7480314960629921,1977.7880435904808,127,0.7804428044280443,1084

+15,0.7241379310344828,33314.86434599152,145,0.6666666666666666,1110.0119178122986,6,0.7363636363636363,-564.124905507706,110,0.7763819095477387,1194

+16,0.7449664429530202,40041.16986266221,149,0.5,15.853120414176374,8,0.7769230769230769,2250.5287698702887,130,0.7764350453172205,1324

+17,0.795774647887324,50441.75925176765,142,0.09090909090909091,-2021.3514247263083,11,0.7758620689655172,1363.2062042449252,116,0.7763888888888889,1440

+18,0.7567567567567568,58252.790501103664,148,0.36363636363636365,46.79459327599034,11,0.8083333333333333,4052.2230580826,120,0.7788461538461539,1560

+19,0.7115384615384616,35154.58693696419,156,0.6666666666666666,1666.3895982081158,9,0.8368794326241135,5683.557568340693,141,0.7836566725455615,1701

+20,0.706766917293233,29838.574412614762,133,0.5,478.89654081649496,4,0.7606837606837606,2907.656908348654,117,0.7821782178217822,1818

+21,0.7619047619047619,43568.27205397414,147,0.4,-248.80626540536468,15,0.7481481481481481,1754.0620352778933,135,0.7798259088581669,1953

+22,0.7901234567901234,63495.04425859108,162,0.6,500.4701962447143,5,0.7913669064748201,2513.065519788506,139,0.7805927342256214,2092

+23,0.7396449704142012,38851.34112204345,169,0.4444444444444444,25.201283025817247,9,0.7753623188405797,2968.4318674558745,138,0.7802690582959642,2230

+24,0.7833333333333333,31115.34323824788,120,0.3333333333333333,-566.9022386348661,6,0.8105263157894737,4754.872992379169,95,0.781505376344086,2325

+25,0.7341772151898734,39120.669093613906,158,0.875,2414.2110238891473,8,0.835820895522388,5220.33426194459,134,0.7844652297681984,2459

+26,0.7973856209150327,62572.113113719824,153,0.3333333333333333,-300.64749333570944,9,0.8035714285714286,3114.099555966168,112,0.7852975495915986,2571

+27,0.74,43655.00922729916,150,0.47058823529411764,594.7447460015246,17,0.7936507936507936,3566.9198998335924,126,0.78568780126066,2697

+28,0.7828947368421053,45026.8397052645,152,0.7142857142857143,1500.2628522217128,7,0.7481481481481481,1084.3524200087268,135,0.7838983050847458,2832

+29,0.7338129496402878,36078.86388836909,139,0.6666666666666666,1317.7853150746523,9,0.7589285714285714,1056.6282104735728,112,0.7829483695652174,2944

+30,0.7573529411764706,38007.991401806285,136,0.6666666666666666,890.51215192565,6,0.7766990291262136,3001.2506227829726,103,0.7827371184771906,3047

+31,0.743421052631579,50489.6403582285,152,0.5555555555555556,670.6301538815751,9,0.7534246575342466,1040.5385060857225,146,0.7813968055120576,3193

+32,0.795774647887324,51562.93794795565,142,0.375,-299.23248941922793,8,0.8110236220472441,3943.773341346605,127,0.7825301204819277,3320

+33,0.7272727272727273,41767.063245811805,165,0.5555555555555556,1870.1602811941848,18,0.7677419354838709,3355.8752452961926,155,0.7818705035971223,3475

+34,0.7402597402597403,45329.90210594355,154,0.5555555555555556,1150.5818248800642,9,0.7804878048780488,2790.558853210896,123,0.7818232351306281,3598

+35,0.8108108108108109,46712.16266022253,148,0.7777777777777778,2278.331285678869,9,0.8235294117647058,4170.624008115628,119,0.7831584611245628,3717

+36,0.7777777777777778,47610.640491850936,153,0.45454545454545453,255.05511497045518,11,0.7465753424657534,995.7790007544827,146,0.7817758219000777,3863

+37,0.7414965986394558,41464.9095506304,147,0.42857142857142855,-208.68833639215154,14,0.813953488372093,3768.466749407962,129,0.7828156312625251,3992

+38,0.7837837837837838,50996.52609834331,148,0.375,-273.37801479385234,8,0.7540983606557377,1487.6839272876532,122,0.7819640252795333,4114

+39,0.8125,47675.89968131513,144,0.7777777777777778,1780.2622920416616,9,0.7867647058823529,1326.860364531094,136,0.7821176470588236,4250

+40,0.8116883116883117,47162.05924845126,154,0.6,518.4042342532484,5,0.8181818181818182,4344.971979249021,132,0.7832040164308535,4382

diff --git a/v1_v2_v3_adaptive_comparison.md b/v1_v2_v3_adaptive_comparison.md

new file mode 100644

index 0000000000000000000000000000000000000000..40dc722655519388d0f89f668759143bc43d7808

--- /dev/null

+++ b/v1_v2_v3_adaptive_comparison.md

@@ -0,0 +1,13 @@

+# V1 vs V2 vs V3-Adaptive Comparison Dashboard

+

+- Trials: **40**

+- V1: win_rate **76.36%**, net_pnl **$44,472.89**, trades **147.93**

+- V2: win_rate **49.35%**, net_pnl **$365.84**, trades **9.45**

+- V3-Adaptive: win_rate **78.41%**, net_pnl **$2,349.07**, trades **109.55**

+- V3-Adaptive learned policy win_rate (rolling): **78.32%**

+- V3-Adaptive policy trades seen: **4382**

+

+## Interpretation

+- V1 usually maximizes participation and raw opportunity capture.

+- V2 usually maximizes strict signal quality but with sparse trade count.

+- V3-Adaptive learns parameter tightness from realized trade outcomes over time.

\ No newline at end of file

diff --git a/v1_v2_v3_comparison.csv b/v1_v2_v3_comparison.csv

new file mode 100644

index
0000000000000000000000000000000000000000..3c427bb53da15f3d0a72be93dcbcf7627f8e3609

--- /dev/null

+++ b/v1_v2_v3_comparison.csv

@@ -0,0 +1,41 @@

+seed,v1_win_rate,v1_net_pnl,v1_trades,v2_win_rate,v2_net_pnl,v2_trades,v3_win_rate,v3_net_pnl,v3_trades

+1,0.7891156462585034,45772.28777931756,147,0.5714285714285714,736.5950265934953,7,0.8666666666666667,2728.6442671177065,45

+2,0.7712418300653595,49354.69653305998,153,0.5,466.5044967666472,8,0.76,455.73816279458697,50

+3,0.7100591715976331,35557.22595198451,169,0.3333333333333333,-1663.5697231146332,18,0.7846153846153846,2252.8309658460785,65

+4,0.8133333333333334,59788.741247466896,150,0.4444444444444444,104.63155918943812,9,0.8297872340425532,2355.2793712304265,47

+5,0.7337278106508875,49642.28889314589,169,0.6,1265.1275420269667,10,0.7407407407407407,1712.3946616715693,54

+6,0.7583892617449665,40290.42753546317,149,0.45454545454545453,-10.481828275223961,11,0.7777777777777778,609.054232656199,45

+7,0.7213114754098361,26250.607016848517,122,0.2727272727272727,-1145.1359702330956,11,0.71875,-514.163379895399,32

+8,0.7481481481481481,42494.23454203876,135,0.5714285714285714,288.90378744305053,7,0.8235294117647058,753.6974449722766,34

+9,0.8074074074074075,42140.74992427282,135,0.6363636363636364,1877.3990842982312,11,0.7555555555555555,583.6846900991804,45

+10,0.7205882352941176,37186.128424909475,136,0.3333333333333333,-371.2778666672966,6,0.75,293.69533909914026,32

+11,0.8068965517241379,52263.664808165704,145,0.0,-1177.7992941021512,4,0.7714285714285715,769.4859993337013,35

+12,0.7801418439716312,44110.473825264344,141,0.35714285714285715,-625.0866076231614,14,0.7551020408163265,731.2338303297292,49

+13,0.7919463087248322,52043.027085619615,149,0.4166666666666667,-211.73701944608183,12,0.6491228070175439,-1274.3127034901408,57

+14,0.7880794701986755,42714.1991954689,151,0.5,-65.92801156388305,10,0.78,1906.2738833403273,50

+15,0.7241379310344828,33314.86434599152,145,0.6666666666666666,1110.0119178122986,6,0.6904761904761905,-389.8621992927656,42

+16,0.7449664429530202,40041.16986266221,149,0.5,15.853120414176374,8,0.7368421052631579,-184.72594645121717,38

+17,0.795774647887324,50441.75925176765,142,0.09090909090909091,-2021.3514247263083,11,0.6666666666666666,-1176.8522363224474,45

+18,0.7567567567567568,58252.790501103664,148,0.36363636363636365,46.79459327599034,11,0.75,2041.3854136843293,52

+19,0.7115384615384616,35154.58693696419,156,0.6666666666666666,1666.3895982081158,9,0.8409090909090909,2248.095094109449,44

+20,0.706766917293233,29838.574412614762,133,0.5,478.89654081649496,4,0.5945945945945946,-1745.1260803819605,37

+21,0.7619047619047619,43568.27205397414,147,0.4,-248.80626540536468,15,0.7931034482758621,1741.0926469929109,58

+22,0.7901234567901234,63495.04425859108,162,0.6,500.4701962447143,5,0.7317073170731707,297.3473507621384,41

+23,0.7396449704142012,38851.34112204345,169,0.4444444444444444,25.201283025817247,9,0.8275862068965517,2508.8790068466915,58

+24,0.7833333333333333,31115.34323824788,120,0.3333333333333333,-566.9022386348661,6,0.7297297297297297,774.5056626183941,37

+25,0.7341772151898734,39120.669093613906,158,0.875,2414.2110238891473,8,0.75,651.7200680357928,44

+26,0.7973856209150327,62572.113113719824,153,0.3333333333333333,-300.64749333570944,9,0.7906976744186046,1885.4735742917546,43

+27,0.74,43655.00922729916,150,0.47058823529411764,594.7447460015246,17,0.7543859649122807,461.26949111036083,57

+28,0.7828947368421053,45026.8397052645,152,0.7142857142857143,1500.2628522217128,7,0.8367346938775511,2014.1972604168404,49

+29,0.7338129496402878,36078.86388836909,139,0.6666666666666666,1317.7853150746523,9,0.8235294117647058,2512.161785434946,51

+30,0.7573529411764706,38007.991401806285,136,0.6666666666666666,890.51215192565,6,0.8571428571428571,1658.8967307595158,35

+31,0.743421052631579,50489.6403582285,152,0.5555555555555556,670.6301538815751,9,0.8444444444444444,1790.3838210195681,45

+32,0.795774647887324,51562.93794795565,142,0.375,-299.23248941922793,8,0.8367346938775511,2054.913421918536,49

+33,0.7272727272727273,41767.063245811805,165,0.5555555555555556,1870.1602811941848,18,0.7901234567901234,2595.3121453423228,81

+34,0.7402597402597403,45329.90210594355,154,0.5555555555555556,1150.5818248800642,9,0.8043478260869565,1593.3563635389146,46

+35,0.8108108108108109,46712.16266022253,148,0.7777777777777778,2278.331285678869,9,0.8421052631578947,1659.1211902808573,38

+36,0.7777777777777778,47610.640491850936,153,0.45454545454545453,255.05511497045518,11,0.7346938775510204,6.046673604102782,49

+37,0.7414965986394558,41464.9095506304,147,0.42857142857142855,-208.68833639215154,14,0.7547169811320755,549.484473723569,53

+38,0.7837837837837838,50996.52609834331,148,0.375,-273.37801479385234,8,0.7073170731707317,-447.11367603384133,41

+39,0.8125,47675.89968131513,144,0.7777777777777778,1780.2622920416616,9,0.7021276595744681,-1099.6838439889689,47

+40,0.8116883116883117,47162.05924845126,154,0.6,518.4042342532484,5,0.7894736842105263,1297.1179314099572,38

diff --git a/v1_v2_v3_comparison.md b/v1_v2_v3_comparison.md

new file mode 100644

index 0000000000000000000000000000000000000000..099184d71e3b85b21407c60d994ff55b2bdf5ce9

--- /dev/null

+++ b/v1_v2_v3_comparison.md

@@ -0,0 +1,11 @@

+# V1 vs V2 vs V3 Comparison Dashboard

+

+- Trials: **40**

+- V1: win_rate **76.36%**, net_pnl **$44,472.89**, trades **147.93**

+- V2: win_rate **49.35%**, net_pnl **$365.84**, trades **9.45**

+- V3 (hybrid): win_rate **76.86%**, net_pnl **$966.52**, trades **46.45**

+

+## V3 design intent

+- Keep V1 participation by allowing all sessions.

+- Apply V2-quality gates adaptively (stricter outside kill zones).

+- Use selective slope alignment (strong setups) and adaptive ATR caps.

\ No newline at end of file

diff --git a/v3_adaptive_training_results.json b/v3_adaptive_training_results.json

new file mode 100644

index 0000000000000000000000000000000000000000..bd1799a491ac5f85ed67bec20a038cb72374e64a

--- /dev/null

+++ b/v3_adaptive_training_results.json

@@ -0,0 +1,282 @@

+[

+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 51463.43953798518,
+ "net_pnl": 1463.4395379851776,
+ "trades": 47,
+ "wins": 38,
+ "losses": 9,
+ "win_rate": 0.8085106382978723,
+ "avg_net_pnl": 31.13701144649314,
+ "profit_factor": 1.5847637734320643,
+ "policy_win_rate": 0.8085106382978723,
+ "policy_trades_seen": 47,
+ "seed": 1000
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 51760.96200613625,
+ "net_pnl": 1760.9620061362511,
+ "trades": 36,
+ "wins": 29,
+ "losses": 7,
+ "win_rate": 0.8055555555555556,
+ "avg_net_pnl": 48.91561128156253,
+ "profit_factor": 2.057755834060217,
+ "policy_win_rate": 0.8072289156626506,
+ "policy_trades_seen": 83,

+ "seed": 1001
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 49526.31280766218,
+ "net_pnl": -473.6871923378203,
+ "trades": 36,
+ "wins": 26,
+ "losses": 10,
+ "win_rate": 0.7222222222222222,
+ "avg_net_pnl": -13.157977564939452,
+ "profit_factor": 0.8288517591362697,
+ "policy_win_rate": 0.7815126050420168,
+ "policy_trades_seen": 119,
+ "seed": 1002
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 50790.72281447938,
+ "net_pnl": 790.7228144793771,
+ "trades": 41,
+ "wins": 32,
+ "losses": 9,
+ "win_rate": 0.7804878048780488,
+ "avg_net_pnl": 19.285922304375053,
+ "profit_factor": 1.3147933673457717,

+ "policy_win_rate": 0.78125,
+ "policy_trades_seen": 160,
+ "seed": 1003
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 48993.852383556004,
+ "net_pnl": -1006.1476164439955,
+ "trades": 68,
+ "wins": 48,
+ "losses": 20,
+ "win_rate": 0.7058823529411765,
+ "avg_net_pnl": -14.796288477117582,
+ "profit_factor": 0.8128697618817533,
+ "policy_win_rate": 0.7587719298245614,
+ "policy_trades_seen": 228,
+ "seed": 1004
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 51150.480843825935,
+ "net_pnl": 1150.4808438259352,
+ "trades": 37,
+ "wins": 29,
+ "losses": 8,
+ "win_rate": 0.7837837837837838,

+ "avg_net_pnl": 31.09407686016041,
+ "profit_factor": 1.5699685951699387,
+ "policy_win_rate": 0.7622641509433963,
+ "policy_trades_seen": 265,
+ "seed": 1005
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 50740.59431998852,
+ "net_pnl": 740.5943199885223,
+ "trades": 48,
+ "wins": 38,
+ "losses": 10,
+ "win_rate": 0.7916666666666666,
+ "avg_net_pnl": 15.429048333094215,
+ "profit_factor": 1.2701480579018185,
+ "policy_win_rate": 0.7667731629392971,
+ "policy_trades_seen": 313,
+ "seed": 1006
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 52527.987102484185,
+ "net_pnl": 2527.987102484185,
+ "trades": 49,
+ "wins": 43,

+ "losses": 6,
+ "win_rate": 0.8775510204081632,
+ "avg_net_pnl": 51.59157352008541,
+ "profit_factor": 2.4225799602597706,
+ "policy_win_rate": 0.7817679558011049,
+ "policy_trades_seen": 362,
+ "seed": 1007
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 50240.810530883464,
+ "net_pnl": 240.8105308834638,
+ "trades": 137,
+ "wins": 102,
+ "losses": 35,
+ "win_rate": 0.7445255474452555,
+ "avg_net_pnl": 1.7577411013391517,
+ "profit_factor": 1.0281389752155188,
+ "policy_win_rate": 0.7715430861723447,
+ "policy_trades_seen": 499,
+ "seed": 1008
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 54231.01052208491,
+ "net_pnl": 4231.010522084907,

+ "trades": 133,
+ "wins": 104,
+ "losses": 29,
+ "win_rate": 0.7819548872180451,
+ "avg_net_pnl": 31.812109188608325,
+ "profit_factor": 1.573820158598336,
+ "policy_win_rate": 0.7737341772151899,
+ "policy_trades_seen": 632,
+ "seed": 1009
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 52614.924838629086,
+ "net_pnl": 2614.9248386290856,
+ "trades": 132,
+ "wins": 99,
+ "losses": 33,
+ "win_rate": 0.75,
+ "avg_net_pnl": 19.81003665628095,
+ "profit_factor": 1.3211761070210641,
+ "policy_win_rate": 0.7696335078534031,
+ "policy_trades_seen": 764,
+ "seed": 1010
+ },
+ {
+ "starting_equity": 50000.0,

+ "ending_equity": 51655.953432861425,
+ "net_pnl": 1655.9534328614245,
+ "trades": 131,
+ "wins": 98,
+ "losses": 33,
+ "win_rate": 0.7480916030534351,
+ "avg_net_pnl": 12.640865899705531,
+ "profit_factor": 1.1963047018555852,
+ "policy_win_rate": 0.7664804469273743,
+ "policy_trades_seen": 895,
+ "seed": 1011
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 51089.870008122816,
+ "net_pnl": 1089.8700081228162,
+ "trades": 113,
+ "wins": 86,
+ "losses": 27,
+ "win_rate": 0.7610619469026548,
+ "avg_net_pnl": 9.644867328520498,
+ "profit_factor": 1.1582693456553752,
+ "policy_win_rate": 0.7658730158730159,
+ "policy_trades_seen": 1008,
+ "seed": 1012
+ },

+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 53884.98067628221,
+ "net_pnl": 3884.980676282212,
+ "trades": 131,
+ "wins": 103,
+ "losses": 28,
+ "win_rate": 0.7862595419847328,
+ "avg_net_pnl": 29.65634104032223,
+ "profit_factor": 1.572614148420424,
+ "policy_win_rate": 0.7682177348551361,
+ "policy_trades_seen": 1139,
+ "seed": 1013
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 53625.751911985644,
+ "net_pnl": 3625.7519119856443,
+ "trades": 101,
+ "wins": 78,
+ "losses": 23,
+ "win_rate": 0.7722772277227723,
+ "avg_net_pnl": 35.89853378203608,
+ "profit_factor": 1.5512557143180228,
+ "policy_win_rate": 0.7685483870967742,
+ "policy_trades_seen": 1240,

+ "seed": 1014
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 48363.40675878433,
+ "net_pnl": -1636.593241215669,
+ "trades": 126,
+ "wins": 90,
+ "losses": 36,
+ "win_rate": 0.7142857142857143,
+ "avg_net_pnl": -12.988835247743404,
+ "profit_factor": 0.8373148574478096,
+ "policy_win_rate": 0.7635431918008785,
+ "policy_trades_seen": 1366,
+ "seed": 1015
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 51744.907009218514,
+ "net_pnl": 1744.9070092185138,
+ "trades": 135,
+ "wins": 105,
+ "losses": 30,
+ "win_rate": 0.7777777777777778,
+ "avg_net_pnl": 12.925237105322324,
+ "profit_factor": 1.2084123935922968,

+ "policy_win_rate": 0.7648234510326449,
+ "policy_trades_seen": 1501,
+ "seed": 1016
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 53419.474787492145,
+ "net_pnl": 3419.474787492145,
+ "trades": 122,
+ "wins": 94,
+ "losses": 28,
+ "win_rate": 0.7704918032786885,
+ "avg_net_pnl": 28.028481864689713,
+ "profit_factor": 1.482026949758292,
+ "policy_win_rate": 0.7652495378927912,
+ "policy_trades_seen": 1623,
+ "seed": 1017
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 50052.033097594685,
+ "net_pnl": 52.03309759468539,
+ "trades": 123,
+ "wins": 91,
+ "losses": 32,
+ "win_rate": 0.7398373983739838,

+ "avg_net_pnl": 0.42303331377792996,
+ "profit_factor": 1.00574023301245,
+ "policy_win_rate": 0.763459335624284,
+ "policy_trades_seen": 1746,
+ "seed": 1018
+ },
+ {
+ "starting_equity": 50000.0,
+ "ending_equity": 50129.10841720019,
+ "net_pnl": 129.10841720018652,
+ "trades": 111,
+ "wins": 82,
+ "losses": 29,
+ "win_rate": 0.7387387387387387,
+ "avg_net_pnl": 1.1631388936953742,
+ "profit_factor": 1.0172307805239011,
+ "policy_win_rate": 0.7619816908992999,
+ "policy_trades_seen": 1857,
+ "seed": 1019
+ }
+]
\ No newline at end of file

diff --git a/v3_policy_state.json b/v3_policy_state.json

new file mode 100644

index 0000000000000000000000000000000000000000..3d8195d22a94bb2d95eb507d0b2f5ca813536133

```
--- /dev/null
+++ b/v3_policy_state.json
@@ -0,0 +1,11 @@
+{
+  "min_score_inside_kz": 25,
+  "min_score_outside_kz": 40,
+  "atr_cap_inside_kz": 45.0,
+  "atr_cap_outside_kz": 35.0,
+  "quick_lock_r_multiple": 0.1,
+  "quick_lock_close_pct": 0.65,
+  "wins": 1415,
+  "losses": 442,
+  "trades_seen": 1857
+}
\ No newline at end of file
diff --git a/v3_policy_state_compare.json b/v3_policy_state_compare.json
new file mode 100644
index 0000000000000000000000000000000000000000..dc720f1c39ab75c3ad5f2a5f5bfaaf45758f6f9f
--- /dev/null
+++ b/v3_policy_state_compare.json
@@ -0,0 +1,11 @@
+{
+  "min_score_inside_kz": 25,
+  "min_score_outside_kz": 40,
+  "atr_cap_inside_kz": 55.0,
```

```
+  "atr_cap_outside_kz": 40.0,
+  "quick_lock_r_multiple": 0.1,
+  "quick_lock_close_pct": 0.65,
+  "wins": 3432,
+  "losses": 950,
+  "trades_seen": 4382
+}
\ No newline at end of file
diff --git a/v3_policy_state_compare2.json b/v3_policy_state_compare2.json
new file mode 100644
index 0000000000000000000000000000000000000000..dc720f1c39ab75c3ad5f2a5f5bfaaf45758f6f9f
--- /dev/null
+++ b/v3_policy_state_compare2.json
@@ -0,0 +1,11 @@
+{
+  "min_score_inside_kz": 25,
+  "min_score_outside_kz": 40,
+  "atr_cap_inside_kz": 55.0,
+  "atr_cap_outside_kz": 40.0,
+  "quick_lock_r_multiple": 0.1,
+  "quick_lock_close_pct": 0.65,
+  "wins": 3432,
+  "losses": 950,
+  "trades_seen": 4382
+}
```

\ No newline at end of file

diff --git a/v3_vs_fusion_comparison.csv b/v3_vs_fusion_comparison.csv

new file mode 100644

index 0000000000000000000000000000000000000000..399c49ab0e22bfcab749290615defe842480c6eb

--- /dev/null

+++ b/v3_vs_fusion_comparison.csv

@@ -0,0 +1,41 @@

+seed,v3_win_rate,v3_net_pnl,v3_trades,fusion_win_rate,fusion_net_pnl,fusion_trades

+1,0.8666666666666667,2728.6442671177065,45,0.8666666666666667,2728.6442671177065,45

+2,0.76,455.73816279458697,50,0.76,575.5984431321122,50

+3,0.7846153846153846,2252.8309658460785,65,0.7846153846153846,2708.4988515437,65

+4,0.8297872340425532,2355.2793712304265,47,0.8297872340425532,2224.8326017595537,47

+5,0.7407407407407407,1712.3946616715693,54,0.7407407407407407,1489.3827595201583,54

+6,0.7777777777777778,609.054232656199,45,0.7777777777777778,609.054232656199,45

+7,0.71875,-560.5266347568249,32,0.71875,-560.5266347568249,32

+8,0.8235294117647058,737.1830108637369,34,0.8235294117647058,737.1830108637369,34

+9,0.7758620689655172,1761.1554021050688,116,0.7758620689655172,1645.0721114909538,116

+10,0.7567567567567568,1619.518062760253,111,0.7567567567567568,1744.8490615454648,111

+11,0.8130841121495327,2633.109034073059,107,0.8130841121495327,2655.4836147319293,107

+12,0.8198198198198198,3504.4683963844727,111,0.8198198198198198,3552.139560072319,111

+13,0.7571428571428571,223.90217200896586,140,0.7571428571428571,3.4727963979094056,140

+14,0.7480314960629921,1977.7880435904808,127,0.7480314960629921,1561.4072471304826,127

+15,0.7363636363636363,-564.124905507706,110,0.7363636363636363,-529.5317104552669,110

+16,0.7769230769230769,2250.5287698702887,130,0.7769230769230769,2310.3919735465024,130

+17,0.7758620689655172,1363.2062042449252,116,0.7758620689655172,891.0571650399652,116

+18,0.8083333333333333,4052.2230580826,120,0.8083333333333333,4077.1092150869663,120

+19,0.8368794326241135,5683.557568340693,141,0.8368794326241135,5931.298152650343,141

+20,0.7606837606837606,2907.656908348654,117,0.7606837606837606,2958.8674791836092,117

+21,0.7481481481481481,1754.0620352778933,135,0.7481481481481481,1608.6550457711564,135

+22,0.7913669064748201,2513.065519788506,139,0.7913669064748201,2625.7962791327736,139

+23,0.7753623188405797,2968.4318674558745,138,0.7753623188405797,2240.7397481896624,138

+24,0.8105263157894737,4754.872992379169,95,0.8105263157894737,4927.651760093067,95

+25,0.835820895522388,5220.33426194459,134,0.835820895522388,5314.270751854128,134

+26,0.8035714285714286,3114.099555966168,112,0.8035714285714286,2870.568082688769,112

+27,0.7936507936507936,3566.9198998335924,126,0.7936507936507936,3372.5755944025877,126

+28,0.7481481481481481,1084.3524200087268,135,0.7481481481481481,1170.917666648238,135

+29,0.7589285714285714,1056.6282104735728,112,0.7589285714285714,810.3177429266361,112

+30,0.7766990291262136,3001.2506227829726,103,0.7766990291262136,3022.3600897005235,103

+31,0.7534246575342466,1040.5385060857225,146,0.7534246575342466,1014.7637375028717,146

+32,0.8110236220472441,3943.773341346605,127,0.8110236220472441,3602.5140749605634,127

+33,0.7677419354838709,3355.8752452961926,155,0.7677419354838709,3745.8366215385468,155

+34,0.7804878048780488,2790.558853210896,123,0.7804878048780488,3183.98140922116,123

+35,0.8235294117647058,4170.624008115628,119,0.8235294117647058,3820.3235203633812,119

+36,0.7465753424657534,995.7790007544827,146,0.7465753424657534,827.0253777132893,146

+37,0.813953488372093,3768.466749407962,129,0.813953488372093,3890.091598502404,129

+38,0.7540983606557377,1487.6839272876532,122,0.7540983606557377,1593.4747497789067,122

+39,0.7867647058823529,1326.860364531094,136,0.7867647058823529,1457.452198437968,136

+40,0.8181818181818182,4344.971979249021,132,0.8181818181818182,4235.04464917416,132

diff --git a/v3_vs_fusion_comparison.md b/v3_vs_fusion_comparison.md

new file mode 100644

index 0000000000000000000000000000000000000000..0a1c7ffe26b01553005d1316b70bf3b42c77b08c

--- /dev/null

+++ b/v3_vs_fusion_comparison.md

@@ -0,0 +1,6 @@

+# V3 Adaptive vs V3 Adaptive Fusion (ICT + StdDev + OTE)

+- Trials: **40**

+- V3 Adaptive: win_rate **78.41%**, net_pnl **$2,349.07**, trades **109.55**

+- Fusion: win_rate **78.41%**, net_pnl **$2,316.22**, trades **109.55**

+

+Fusion strategy enforces balanced ICT and StdDev+OTE agreement with equal-weight blended scoring.

\ No newline at end of file

diff --git a/v3_vs_stdote_comparison.csv b/v3_vs_stdote_comparison.csv

new file mode 100644

index 0000000000000000000000000000000000000000..8d7fc572b9bab3fb81f215ef7b9d2d8837f762c9

--- /dev/null

+++ b/v3_vs_stdote_comparison.csv

@@ -0,0 +1,41 @@

+seed,v3_win_rate,v3_net_pnl,v3_trades,stdote_win_rate,stdote_net_pnl,stdote_trades

+1,0.8666666666666667,2728.6442671177065,45,0.8703703703703703,6476.713021640317,54

+2,0.76,455.73816279458697,50,0.8666666666666667,4871.474568284029,45

+3,0.7846153846153846,2252.8309658460785,65,0.8292682926829268,5796.0389586296515,82

+4,0.8297872340425532,2355.2793712304265,47,0.8405797101449275,7031.218415537078,69

+5,0.7407407407407407,1712.3946616715693,54,0.7971014492753623,2743.6178883897082,69

+6,0.7777777777777778,609.054232656199,45,0.8888888888888888,3975.9822070627706,54

+7,0.71875,-560.5266347568249,32,0.85,1451.1079061085911,20

+8,0.8235294117647058,737.1830108637369,34,0.9090909090909091,4818.502654530312,44

+9,0.7758620689655172,1761.1554021050688,116,0.875,2770.53097315169,56

+10,0.7567567567567568,1619.518062760253,111,0.85,1396.460195158652,40

+11,0.8130841121495327,2633.109034073059,107,0.8478260869565217,1686.124251785077,46

+12,0.8198198198198198,3504.4683963844727,111,0.8709677419354839,3307.2382099531387,62

+13,0.7571428571428571,223.90217200896586,140,0.9056603773584906,4271.379040382963,53

+14,0.7480314960629921,1977.7880435904808,127,0.8076923076923077,1433.259008111927,52

+15,0.7363636363636363,-564.124905507706,110,0.8064516129032258,789.1702611307846,31

+16,0.7769230769230769,2250.5287698702887,130,0.7608695652173914,808.919780924115 9,46

+17,0.7758620689655172,1363.2062042449252,116,0.7936507936507936,2766.0631914936675,63

+18,0.8083333333333333,4052.2230580826,120,0.7959183673469388,69.89160682933289,49

+19,0.8368794326241135,5683.557568340693,141,0.8714285714285714,4110.588862415534,70

+20,0.7606837606837606,2907.656908348654,117,0.8846153846153846,2282.75689895534,52

+21,0.7481481481481481,1754.0620352778933,135,0.8148148148148148,3427.6210293423355,54

+22,0.7913669064748201,2513.065519788506,139,0.8333333333333334,4304.219396266853,72

+23,0.7753623188405797,2968.4318674558745,138,0.8059701492537313,38.38087301250198,67

+24,0.8105263157894737,4754.872992379169,95,0.84375,244.8861326801125,32

+25,0.835820895522388,5220.33426194459,134,0.8392857142857143,1843.2930386983207,56

+26,0.8035714285714286,3114.099555966168,112,0.7857142857142857,-34.48996652070491,56

+27,0.7936507936507936,3566.9198998335924,126,0.8771929824561403,4285.151032273352,57

+28,0.7481481481481481,1084.3524200087268,135,0.8125,519.7610537414585,48

+29,0.8235294117647058,2512.161785434946,51,0.8235294117647058,1915.6967923275806,68

+30,0.8571428571428571,1658.8967307595158,35,0.9696969696969697,5300.994233218444,33

+31,0.8444444444444444,1787.0628088195372,45,0.8703703703703703,3381.005922053664,54

+32,0.8367346938775511,1944.072331491625,49,0.8490566037735849,5154.317954880389,53

+33,0.7677419354838709,3355.8752452961926,155,0.8,1958.6967952077248,90

+34,0.7804878048780488,2790.558853210896,123,0.8032786885245902,1432.4734509365226,61

+35,0.8235294117647058,4170.624008115628,119,0.8490566037735849,6078.278075643393,53

+36,0.7465753424657534,995.7790007544827,146,0.85,3187.8256887765747,60

+37,0.813953488372093,3768.466749407962,129,0.8387096774193549,1475.0587089059263,62

+38,0.7540983606557377,1487.6839272876532,122,0.8727272727272727,5161.665388091569,55

+39,0.7867647058823529,1326.860364531094,136,0.7708333333333334,754.5222066142014,48

+40,0.8181818181818182,4344.971979249021,132,0.75,-304.83459464870975,64

diff --git a/v3_vs_stdote_comparison.md b/v3_vs_stdote_comparison.md

new file mode 100644

index 0000000000000000000000000000000000000000..5ac7dcd791bcf0e1f62813a7c4ffa1271c3e290a

--- /dev/null

+++ b/v3_vs_stdote_comparison.md

@@ -0,0 +1,6 @@

+# V3 Adaptive vs StdDev+OTE Adaptive

+- Trials: **40**

+- V3 Adaptive: win_rate **79.07%**, net_pnl **$2,320.57**, trades **101.85**

+- StdDev+OTE: win_rate **83.95%**, net_pnl **$2,824.54**, trades **55.00**

+

+StdDev+OTE version removes ICT OB/FVG/MSS confluence logic and replaces it with statistical deviation + OTE retracement confluence.

\ No newline at end of file

diff --git a/winrate_sim_dashboard.md b/winrate_sim_dashboard.md

new file mode 100644

index 0000000000000000000000000000000000000000..f57c1a31c0146630206bb85a0a4d3b2f8185aabe

--- /dev/null

+++ b/winrate_sim_dashboard.md

@@ -0,0 +1,21 @@

+# Winrate Simulation Dashboard

+

+## Aggregate Results

+- Trials: **40**

+- Avg trades per trial: **18.07**

+- Avg win rate: **86.46%**

+- Median win rate: **85.71%**

+- Win rate range: **65.52% → 100.00%**

+- Avg net PnL: **$6,486.92**

+- Median net PnL: **$5,976.97**

+- Net PnL range: **$3,316.85 → $11,129.89**

+- Avg profit factor (finite only): **13.86**

+- Trials with no losses (PF=∞): **3**

+

+## Best / Worst Trial

+- Best seed #31: net=$11,129.89, win_rate=100.00%, trades=26

+- Worst seed #15: net=$3,316.85, win_rate=73.33%, trades=15

+

+## Notes

+- This is a synthetic market simulation (not live/exchange historical feed).

+- Use `winrate_sim_results.csv` to inspect per-trial outcomes.

\ No newline at end of file

diff --git a/winrate_sim_results.csv b/winrate_sim_results.csv

new file mode 100644

index 0000000000000000000000000000000000000000..ee51582625d6cb3a967eafd49728dff17c3223de

--- /dev/null

+++ b/winrate_sim_results.csv

@@ -0,0 +1,41 @@

+seed,trades,wins,losses,win_rate,net_pnl,profit_factor

+1,19,15,4,0.7894736842105263,5193.0668706668075,6.270777846040726

+2,15,12,3,0.8,6351.29032694349,7.117690479854514

+3,29,19,10,0.6551724137931034,4487.634625803621,2.802204960024655

+4,12,11,1,0.9166666666666666,4201.478995359139,22.096116828782336

+5,19,16,3,0.8421052631578947,5618.855705004535,9.898902589150048

+6,17,17,0,1.0,6232.379715519855,inf

+7,11,10,1,0.9090909090909091,4849.649121416223,14.002299681977036

+8,14,12,2,0.8571428571428571,6036.441550253585,11.166917214543675

+9,15,13,2,0.8666666666666667,5315.108804673728,11.597056242510721

+10,16,12,4,0.75,9093.815077126492,10.724819260329301

+11,18,16,2,0.8888888888888888,8761.810187493538,13.299719416585372

+12,15,14,1,0.9333333333333333,5739.457644128488,27.512577691750028

+13,22,19,3,0.8636363636363636,9211.480461358806,13.07212200450581

+14,18,17,1,0.9444444444444444,6107.258472140529,23.031207208887935

+15,15,11,4,0.7333333333333333,3316.8450279609533,3.599602450684941

+16,18,15,3,0.8333333333333334,4395.370262352735,6.45424497337657

+17,20,19,1,0.95,9468.444617987116,51.913034627185844

+18,16,15,1,0.9375,5156.973721040995,25.891844672921213

+19,17,15,2,0.8823529411764706,5313.672701233445,12.131111679140945

+20,14,11,3,0.7857142857142857,3864.566331825481,5.41719803970448

+21,19,17,2,0.8947368421052632,8311.25452777851,12.834685303940628

+22,21,18,3,0.8571428571428571,9129.889183021369,9.778844553902804

+23,11,10,1,0.9090909090909091,3502.076234997221,17.95724363990531

+24,14,11,3,0.7857142857142857,3612.1908719822386,5.617386242290614

+25,10,10,0,1.0,5740.767598799503,inf

+26,26,22,4,0.8461538461538461,8685.1747765053,7.733649834441981

+27,16,13,3,0.8125,4216.679814368108,5.836352722614273

+28,15,14,1,0.9333333333333333,5680.74945673201,22.902861383627638

+29,23,19,4,0.8260869565217391,7159.670110394363,8.196151486010944

+30,20,17,3,0.85,8764.84414784456,13.251676431423883

+31,26,26,0,1.0,11129.88728696444,inf

+32,17,14,3,0.8235294117647058,5917.505216822319,8.61366879414556

+33,33,26,7,0.7878787878787878,8870.318867733731,5.746230344896079

+34,20,17,3,0.85,8890.35133739315,10.116779515378278

+35,14,12,2,0.8571428571428571,4126.359882306642,8.759047001691995

+36,23,19,4,0.8260869565217391,7632.364473199777,8.911490879720922

+37,22,18,4,0.8181818181818182,6751.296272031323,7.808278978197636

+38,20,19,1,0.95,10166.956811027267,48.73761192110842

+39,17,16,1,0.9411764705882353,5792.238478238942,22.259864166451376

+40,16,14,2,0.875,6680.762635790379,9.80804291446048